

Searching Part-4

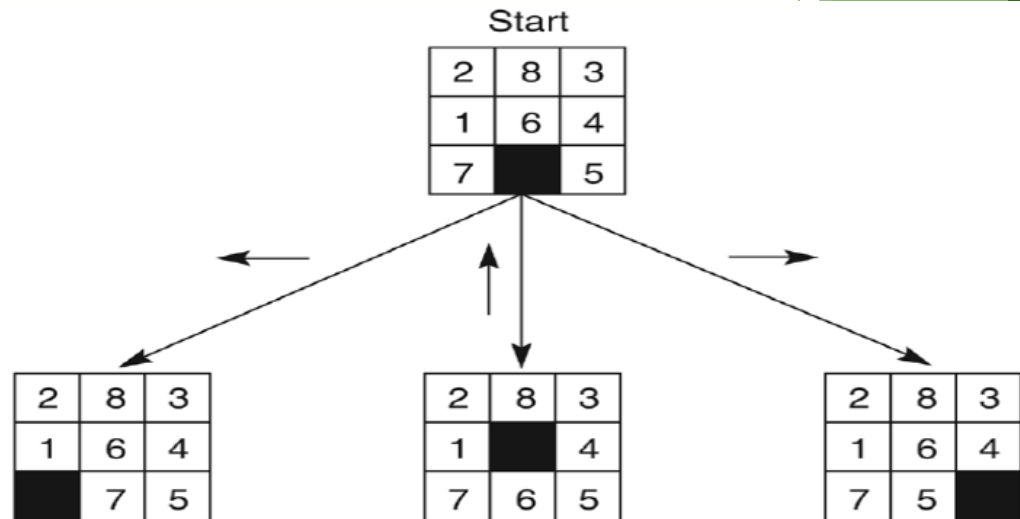


Kuliah : Maret 2024

Review Kasus A* dalam 8-Puzzle

$$g(n) = 0$$

$$g(n) = 1$$



Values of $f(n)$ for each state,

6

4

6

where:

$$f(n) = g(n) + h(n),$$

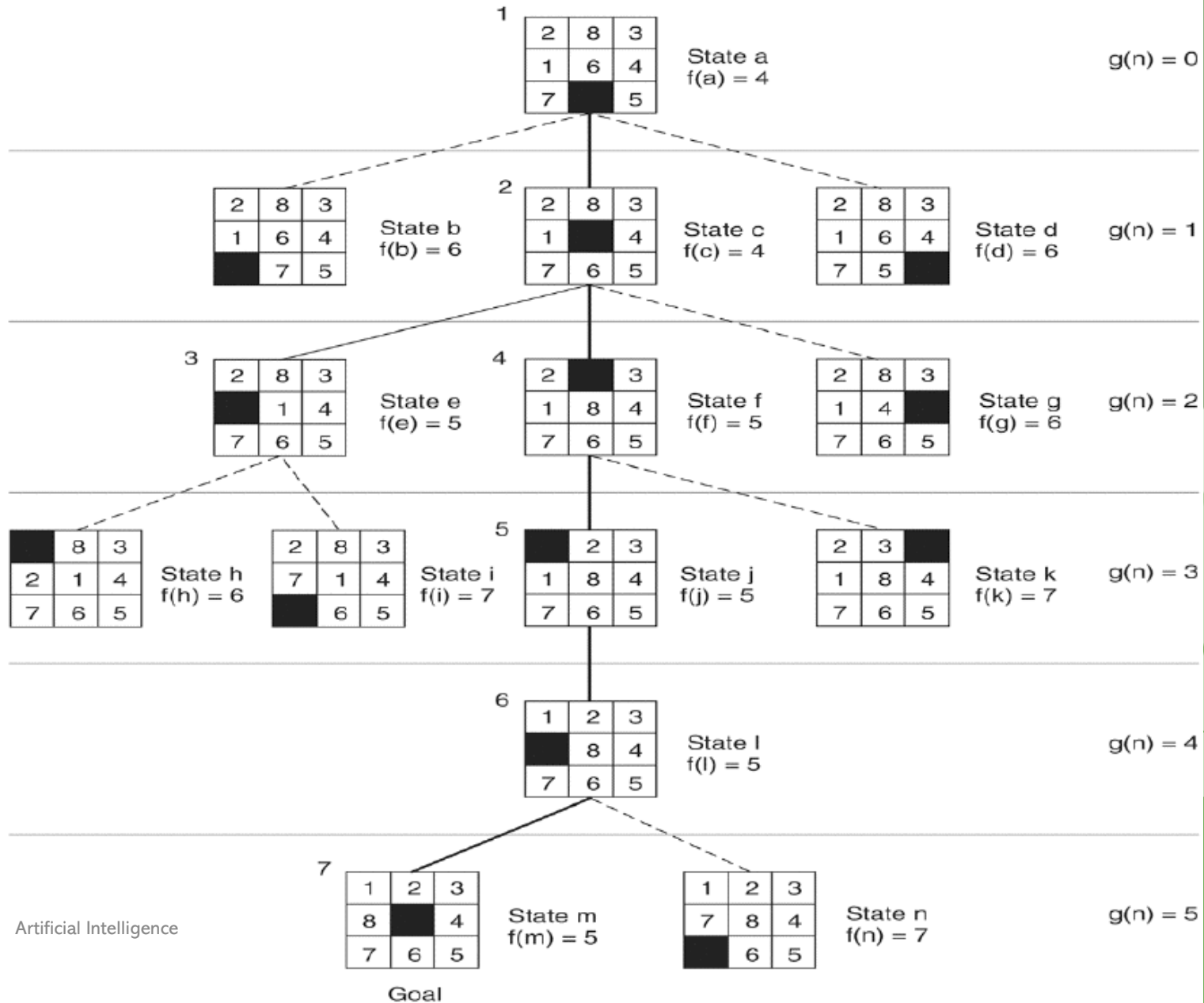
$g(n)$ = actual distance from n
to the start state, and

$h(n)$ = number of tiles out of place.

1	2	3
8		4
7	6	5

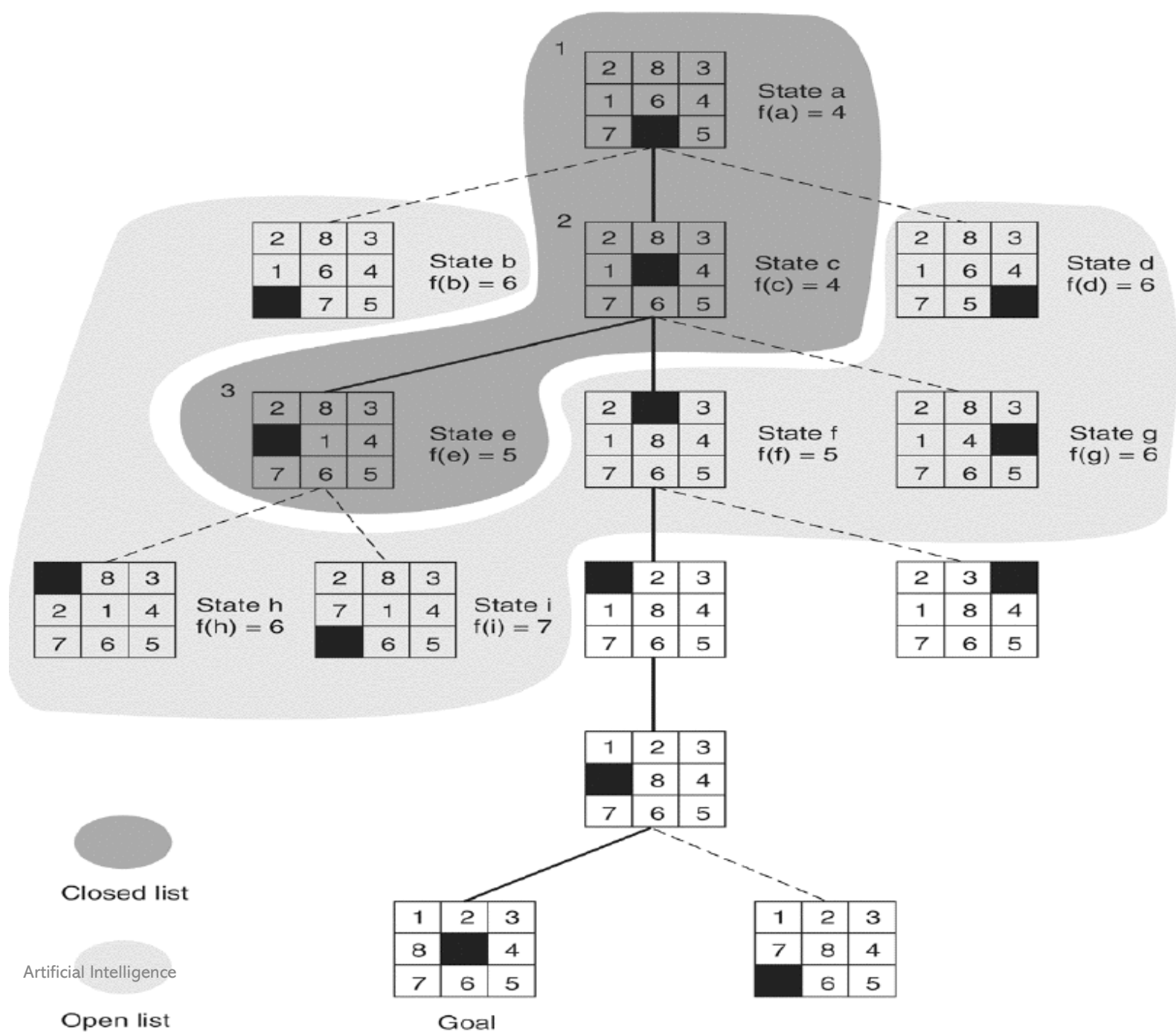
Goal

Level of search
 $g(n) =$



Proses Algoritma A*

1. **OPEN** =[A4]; **CLOSED**-[]
2. **OPEN**=[C4,B6,D6]; **CLOSED**=[A4]
3. **OPEN**=[E5,F5,B6,D6,G6]; **CLOSED**=[A4,C4]
4. **OPEN**=[F5,H6,B6,D6,G6,I7]; **CLOSED**=[A4,C4,E5]
5. **OPEN**=[J5,H6,B6,D6,G6,K7,I7]; **CLOSED**=[A4,C4,E5,F5]
6. **OPEN**=[L5,H6,B6, D6,G6,K7,I7]; **CLOSED**=[A4,C4,E5,F5,J5]
7. **OPEN**=[M5,H6,B6,D6,G6,N7,K7,I7];**CLOSED**=[A4,C4,E5,F5,J5,L5]
8. **SUCCESS**, M=GOAL; **CLOSED**=[A4,C4,E5,F5,J5,L5,M]



Studi kasus



Initial State



Goal State

- ▶ Buat kelompok dengan anggota 3 mhs
- ▶ Kerjakan, dalam waktu 45 menit
- ▶ Di kerjakan di kertas kemudian di kumpulkan

Iterative Deepening A (IDA*)*

- ▶ A* yang iteratif dengan batasan berupa *F-limit*
- ▶ **Complete** dan **optimal**
- ▶ Iteratif → *time complexity*-nya tinggi
- ▶ Keuntungan: *space complexity* sangat rendah
- ▶ Sesuai untuk *mobile systems*
- ▶ F-limit → Nilai gabungan antara biaya sebenarnya dan biaya perkiraan
- ▶ Pada IDA* → setiap iterasi akan mengembalikan nilai **F-limit** baru yang akan digunakan sebagai batasan untuk pencarian berikutnya

function IDA*(*masalah*) **returns** *solusi*

inputs: *masalah*, sebuah *masalah*

local variables: *f-limit*, batasan *f-Cost* saat ini
root, sebuah simpul

root $\leftarrow$ MAKE-SIMPUL(INITIAL-STATE[*masalah*])

f-limit $\leftarrow$ *f-Cost*(*root*)

loop do

solusi, *f-limit* $\leftarrow$ DFS-CONTOUR(*root*, *f-limit*)

if *solusi* bukan *null* **then** return *solusi*

if *f-limit* sama dengan *INFINITE* **then** return gagal

end

function DFS-CONTOUR(*simpul*, *f-limit*) **returns** *solusi*, batasan *f-Cost* yang baru

inputs: *simpul*, sebuah simpul

f-limit, batasan *f-Cost* saat ini

local variables: *next-f*, batasan *f-Cost* untuk penelusuran berikutnya, pada awalnya variabel ini di beri nilai tak hingga (*INFINITE*)

if *f-Cost*[*simpul*] > *f-limit* **then return** *null*, *f-Cost*[*simpul*]

if GOAL-TEST[problem](STATE[*simpul*]) **then return** *simpul*, *f-limit*

for setiap simpul *s* yang menjadi SUCCESSOR(*simpul*) **do**

solusi, *new-f* $\leftarrow$ DFS-CONTOUR(*s*, *f-limit*)

if *solusi* bukan *null* **then return** *solusi*, *f-limit*

next-f $\leftarrow$ MIN(*next-f*, *new-f*)

end

return *null*, *next-f*

Alternatif Algoritma

Function `depth_first_search(n, limit)`

If $f(n) > \text{limit}$ **Then Return** $f(n)$

If $h(n) = 0$ **Then** BERHASIL

Return nilai terendah dari `depth_first_search(ni, limit)` bagi semua *successor* ni dari n

end

Procedure `IDA*(n)`

`limit = h(n)`

repeat

`limit = depth_first_search(n, limit)`

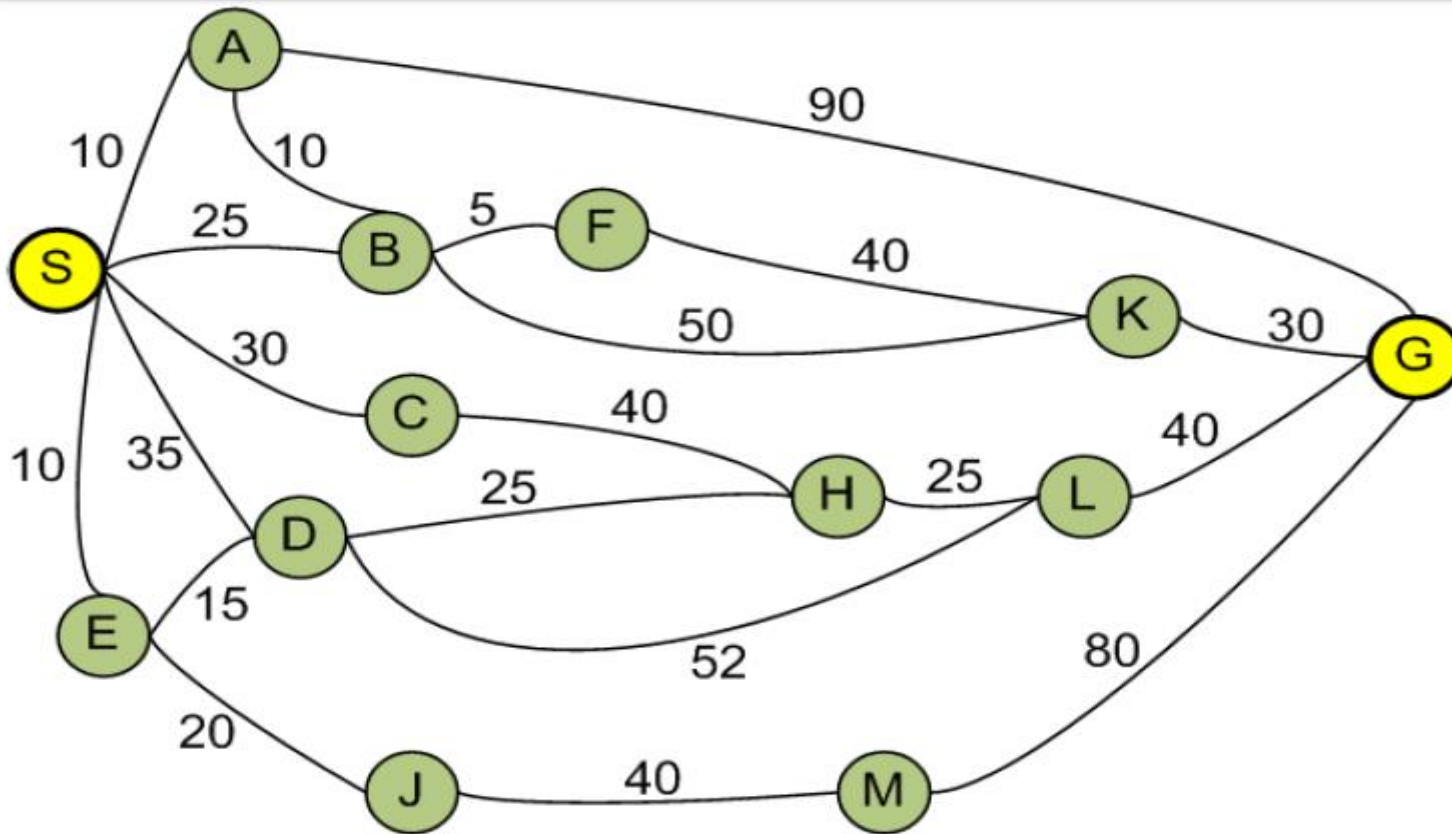
until BERHASIL

end

Penjelasan IDA*

- ▶ IDA* merupakan gabungan *iterative deepening depth first search* dengan A*.
- ▶ Pencarian dilakukan secara iteratif. Batasan/limit pencarian didasarkan pada nilai $f(n)$.
- ▶ Dimulai dari status awal, tentukan suatu limit awal.
- ▶ Bangkitkan semua node dari status awal tersebut. Hitung $f(n)$ untuk setiap anak yang dibangkitkan. Mana $f(n)$ terkecil?
- ▶ Apakah $f(n)$ terkecil lebih kecil dari limit?
 - ▶ **IYA**. Bangkitkan semua anak dari node dengan $f(n)$ terkecil tersebut. Lanjutkan.
 - ▶ **TIDAK**. Jadikan $f(n)$ tersebut sebagai limit baru. Ulangi pencarian dari status awal (**RESTART**).
- ▶ Boros waktu tetapi tetap memberikan solusi terbaik.

Contoh Kasus



Jarak Garis Lurus Kota n ke Kota G

n	S	A	B	C	D	E	F	G	H	J	K	L	M
h(n)	80	80	60	70	85	74	70	0	40	100	30	20	70

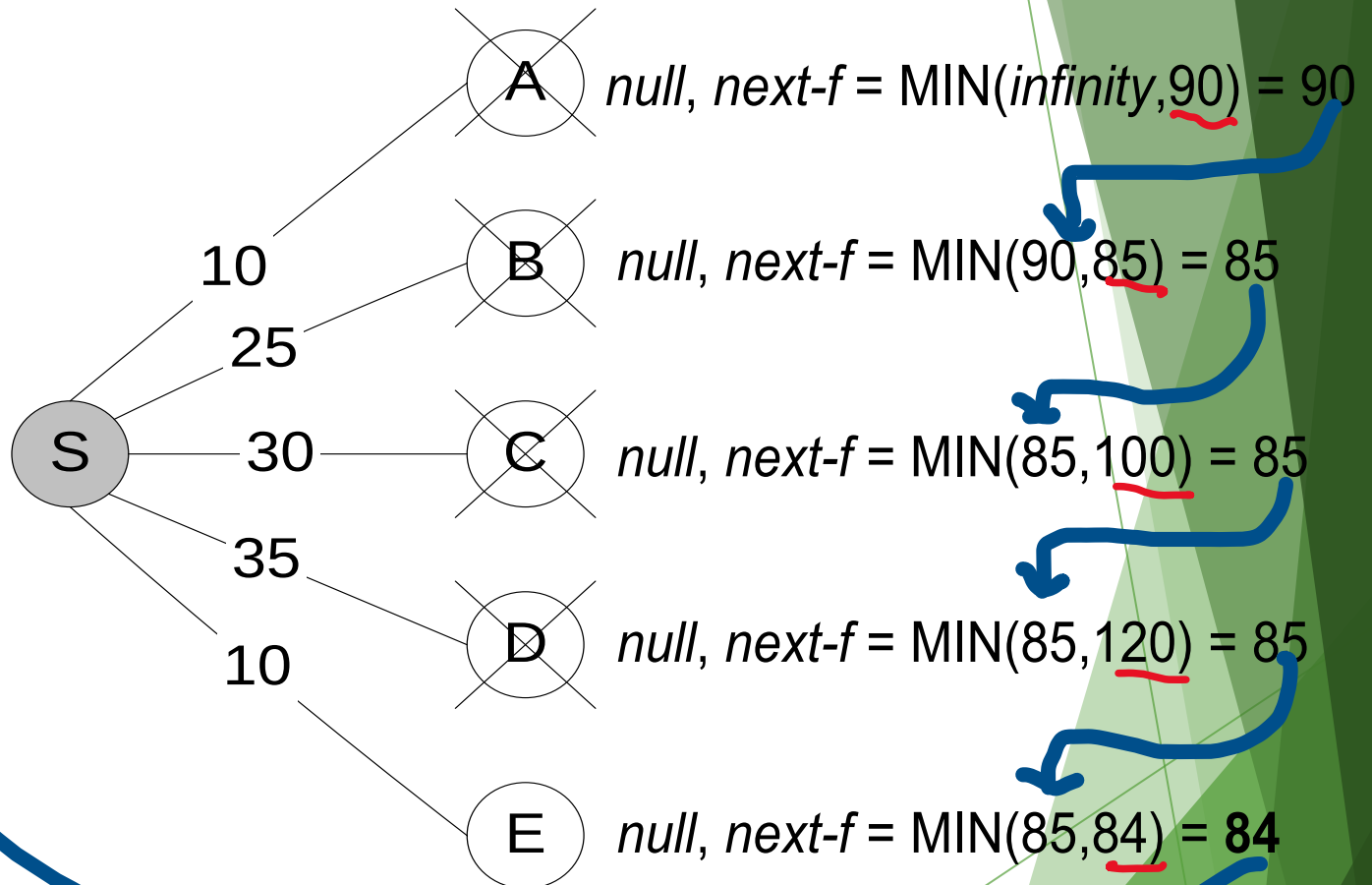
Langkah 1:

inputs:

- *node* = S
- *f-limit* = 80

returns:

- *null*
- *next-f* = 84



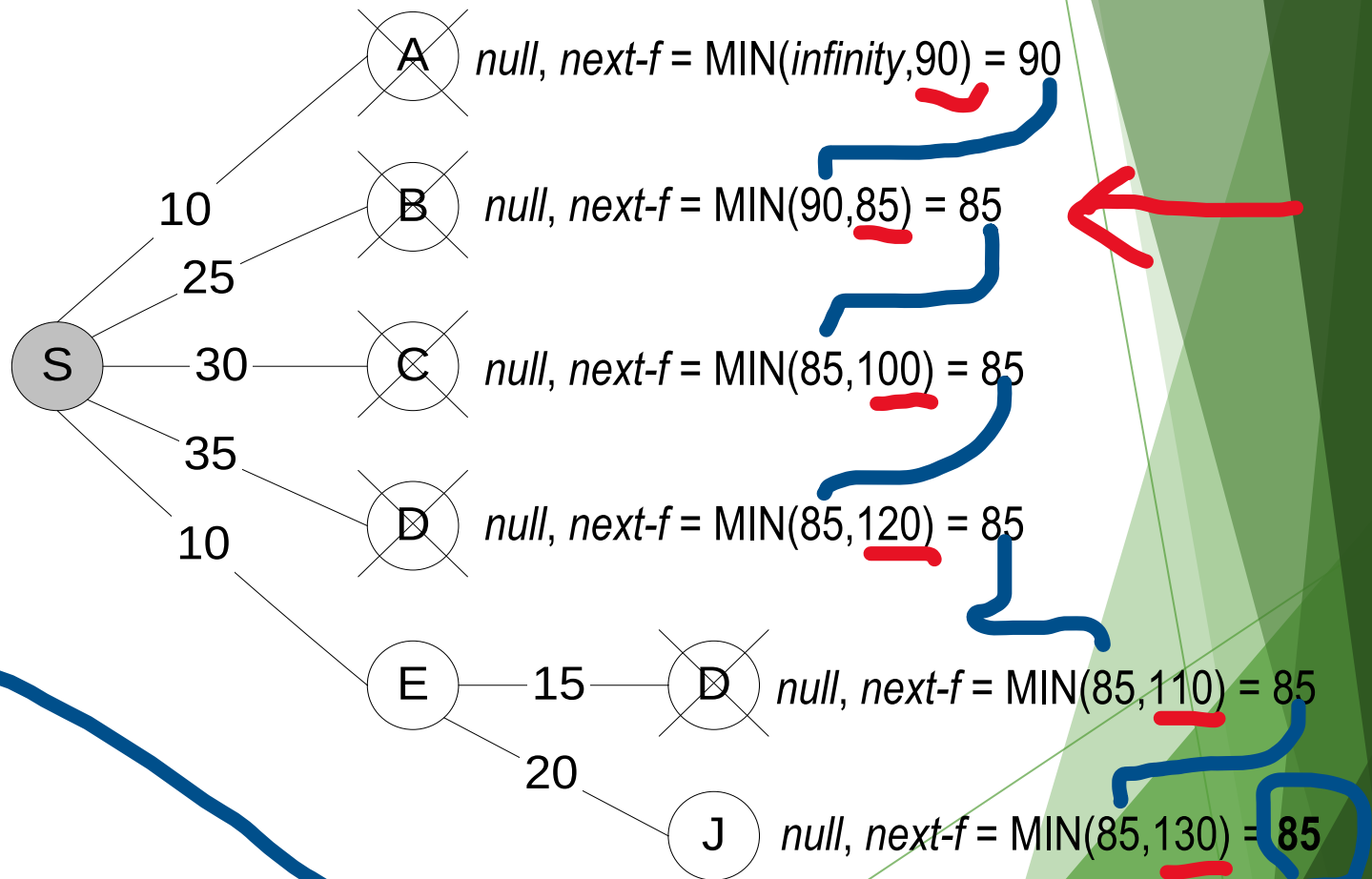
Langkah 2:

inputs:

- *node* = S
- *f-limit* = 84

returns:

- *null*
- *next-f* = 85



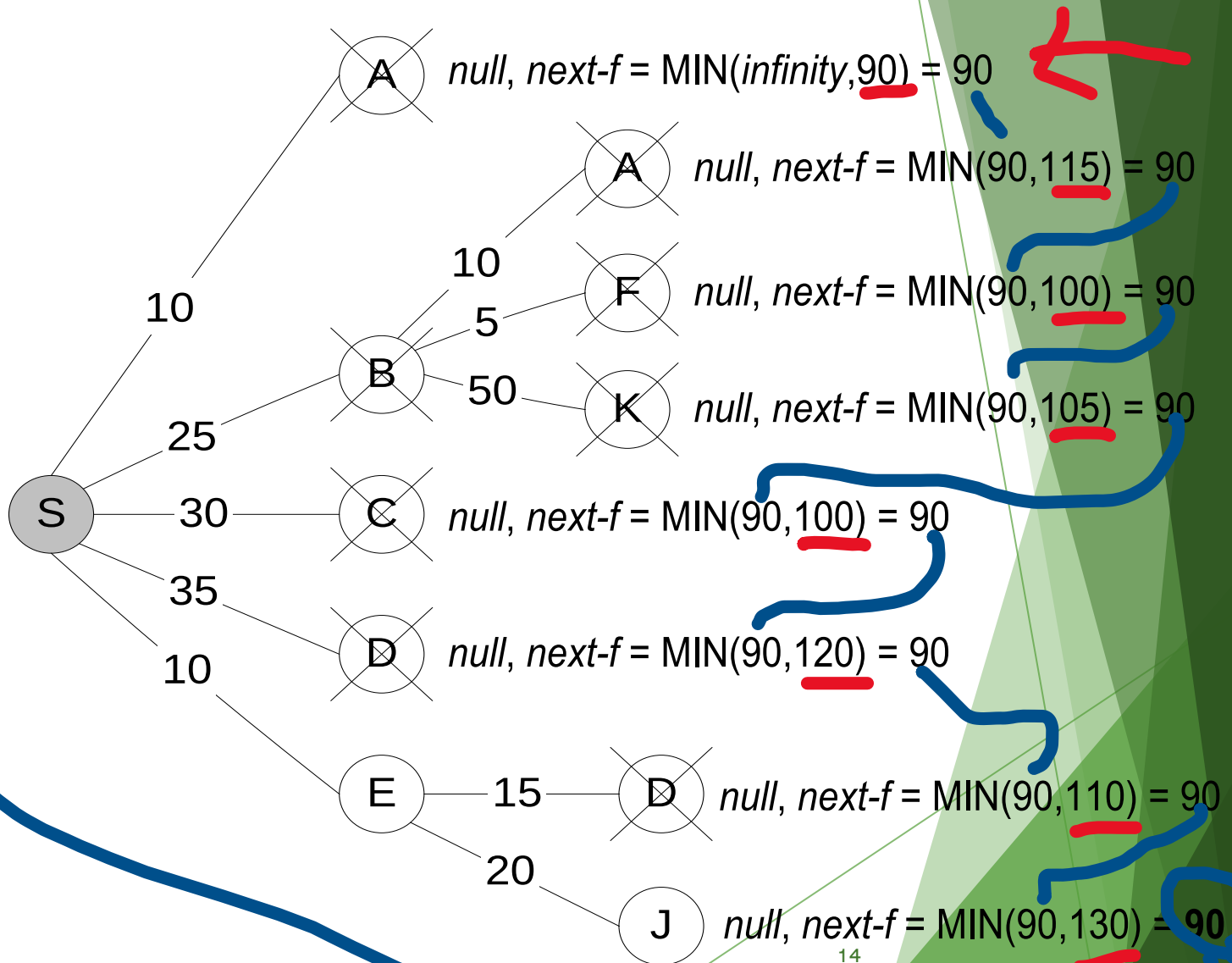
Langkah 3:

inputs:

- *node* = S
- *f-limit* = 85

returns:

- *null*
- *next-f* = 90



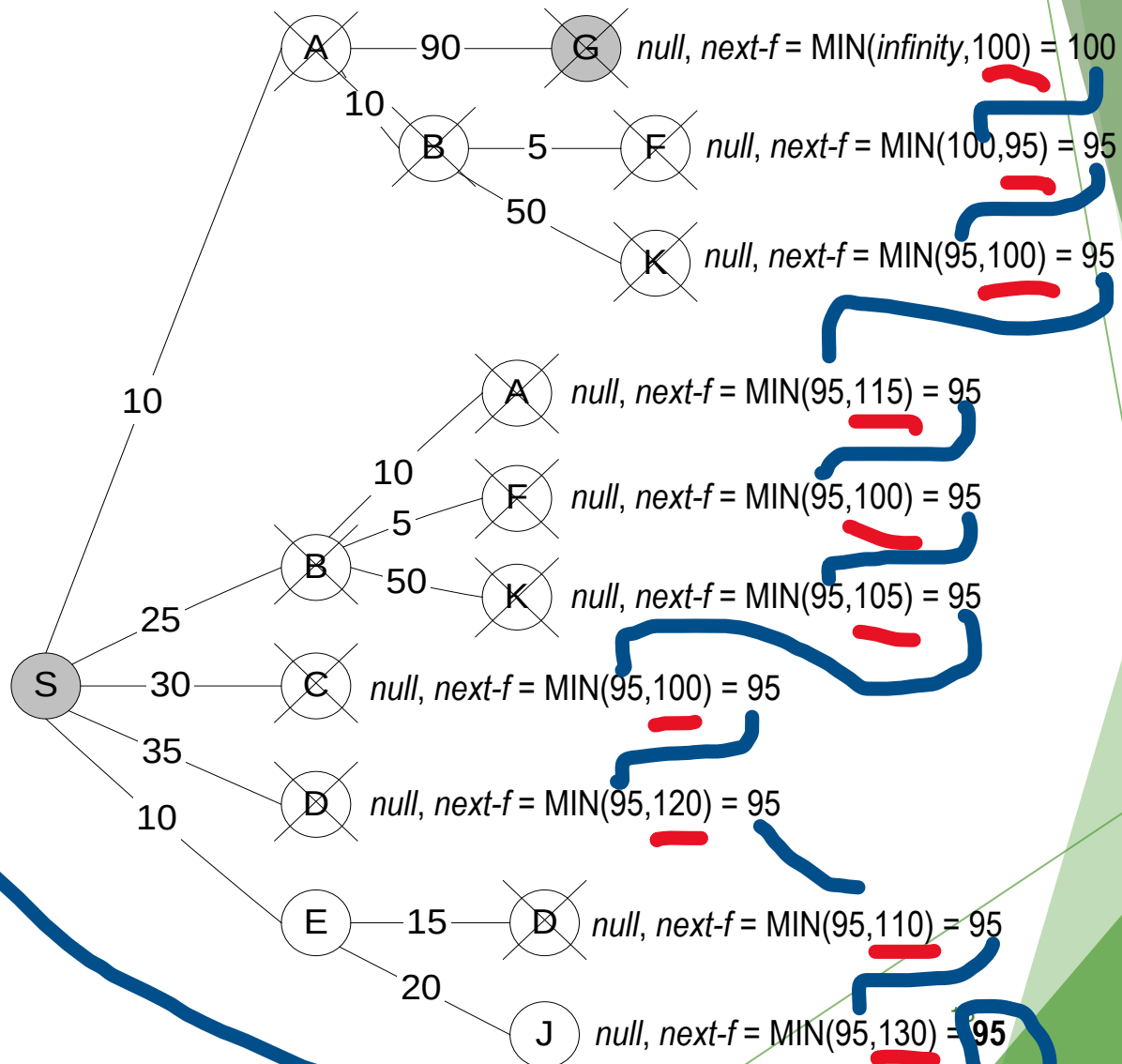
Langkah 4:

inputs:

- *node* = S
- *f-limit* = 90

returns:

- *null*
- *next-f* = 95



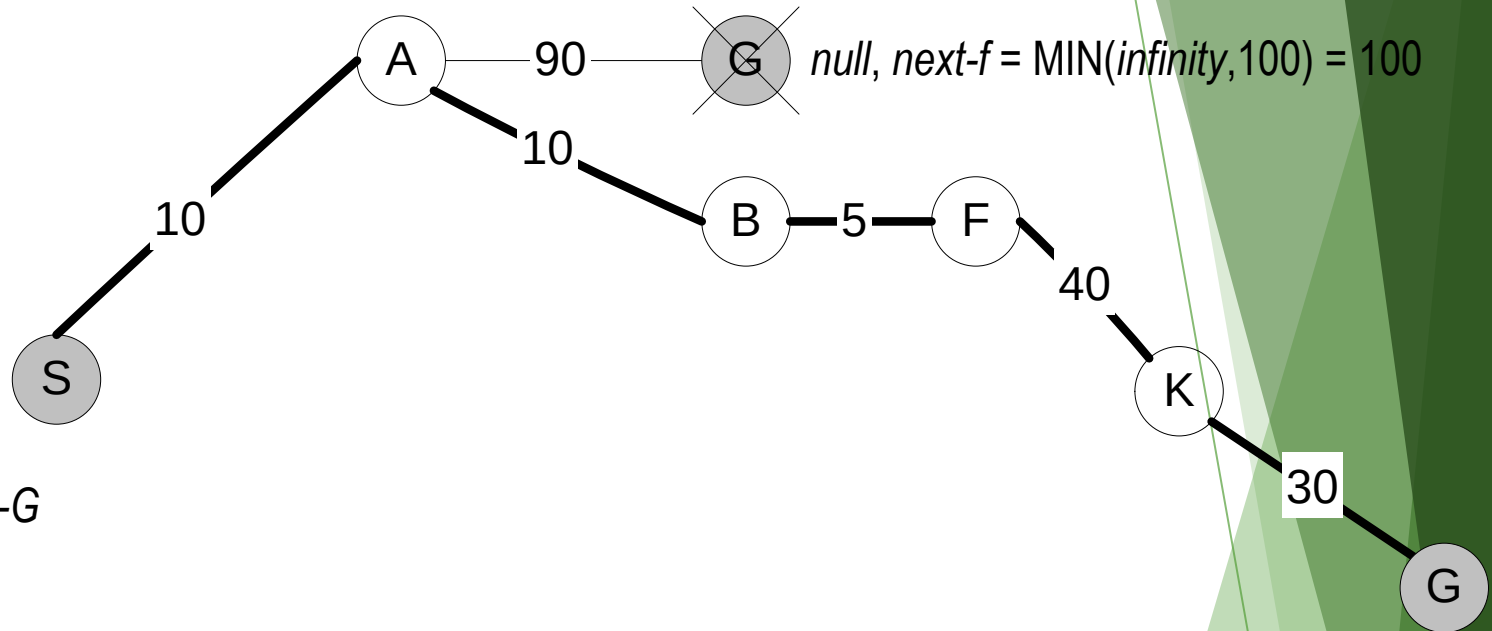
Langkah 5:

inputs:

- *node* = S
- *f-limit* = 95

returns:

- *solusi*: S-A-B-F-K-G
- *next-f* = 95



Simplified Memory-Bounded A* (SMA*)

- ▶ IDA* yang hanya mengingat satu *f-limit*
- ▶ SMA* **mengingat *f-Cost*** dari setiap iterasi sampai sejumlah simpul yang ada di dalam memori.
- ▶ Jika memori komputer hanya mampu menyimpan 100 simpul, maka kita bisa membatasi proses pencarian sampai level 99.
- ▶ Memory yang tersedia sangat terbatas
- ▶ Menghindari ekspansi terhadap node-node yang telah diekspan sebelumnya
- ▶ Hanya menyimpan node yang paling menjanjikan
- ▶ Harus mengingat “sesuatu” untuk membangkitkan kembali yang tak disimpan dengan cepat
- ▶ Jika memory penuh dan masih perlu membangkitkan node tambahan:
 - ▶ Hapus node daun dengan nilai **f tertinggi dan Terdangkal**
 - ▶ Ingat nilai **f** dari anak terbaik (yang tak disimpan) pada setiap node induk.

function SMA*(*masalah*) **returns** *solusi*

inputs: *masalah*, sebuah masalah

local variables: *Queue*, antrian nodes yang terurut berdasarkan *f-cost*

Queue $\leftarrow$ MAKE-QUEUE({MAKE-SIMPUL(INITIAL-STATE[*masalah*])})

loop do

if *Queue* kosong **then return** gagal

n $\leftarrow$ simpul di *Queue* yang memiliki *f-cost* terkecil dan level terdalam

if GOAL-TEST(*n*) **then return** sukses

s $\leftarrow$ NEXT-SUCCESSOR(*n*)

if *s* bukan goal dan levelnya sudah maksimum **then**

f(s) $\leftarrow$ INFINITE

else

f(s) $\leftarrow$ MAX(*f(n)*, *g(s)* + *h(s)*)

end

if semua suksesor dari *n* sudah dibangkitkan **then**

 Ganti *f-cost* pada *n* dengan nilai *f(s)* yang terkecil. Gantikan nilai *f(s)* terkecil ini ke semua *ancestors* dari *n* (ayah, kakek, dan seterusnya ke atas) kecuali *ancestors* yang memiliki *f-cost* lebih kecil daripada *f(s)* terkecil itu.

if semua SUCCESSORS(*n*) sudah di memori **then**

 Keluarkan *n* dari *Queue* (tetapi tidak dihapus secara fisik di memori)

if memori penuh **then**

 Hapus simpul terburuk yang memiliki *f-cost* terbesar dan level terdangkal (artinya: jika terdapat lebih dari satu simpul dengan *f-cost* terbesar, maka dipilih simpul yang levelnya terdangkal).

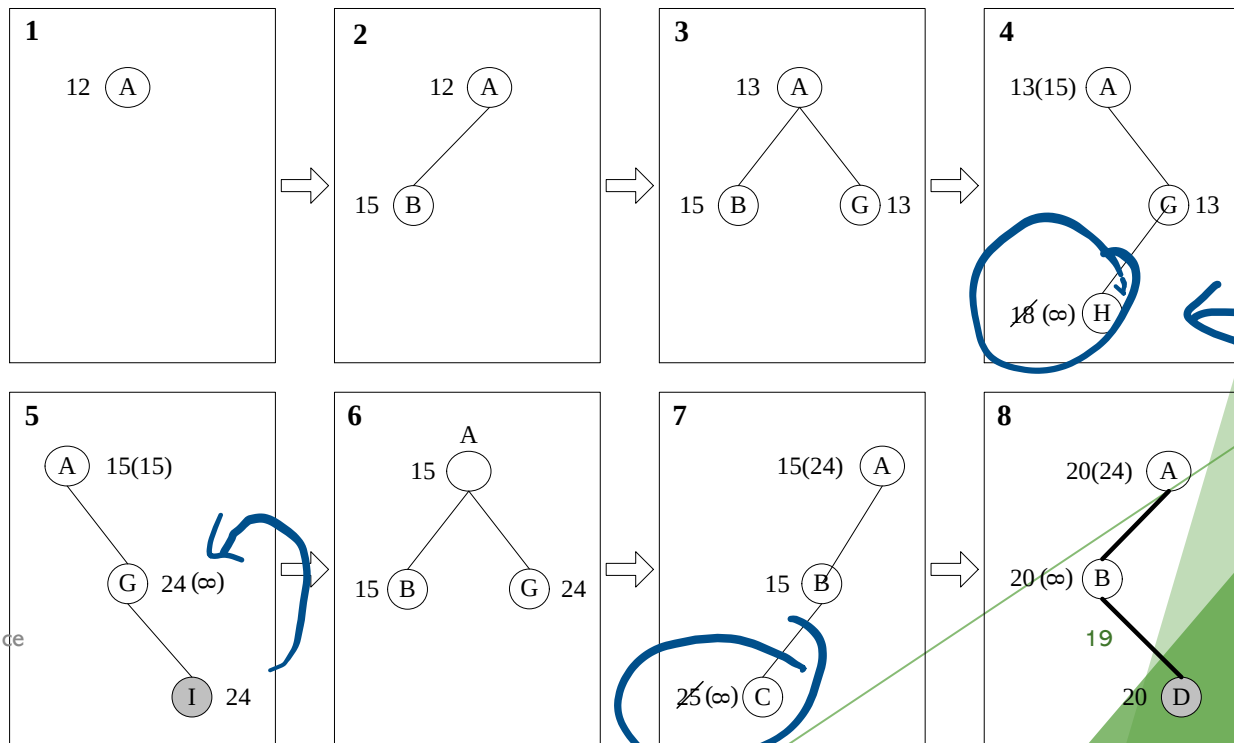
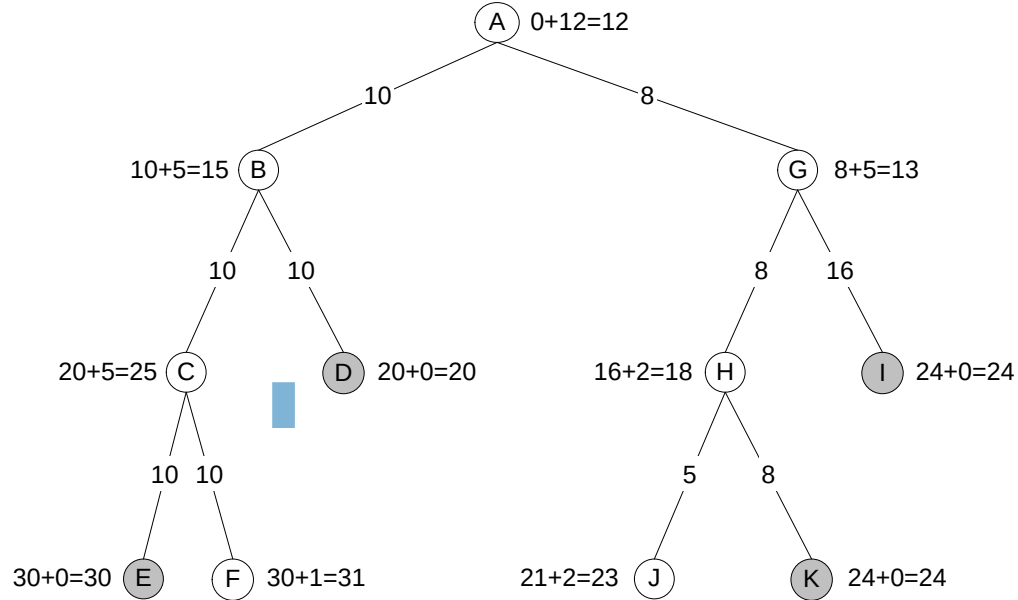
 Keluarkan simpul terburuk tersebut dari daftar suksesor *parent*-nya.

 Masukkan *parent* dari simpul terburuk tersebut ke *Queue* jika *parent* tersebut tidak ada di *Queue*.

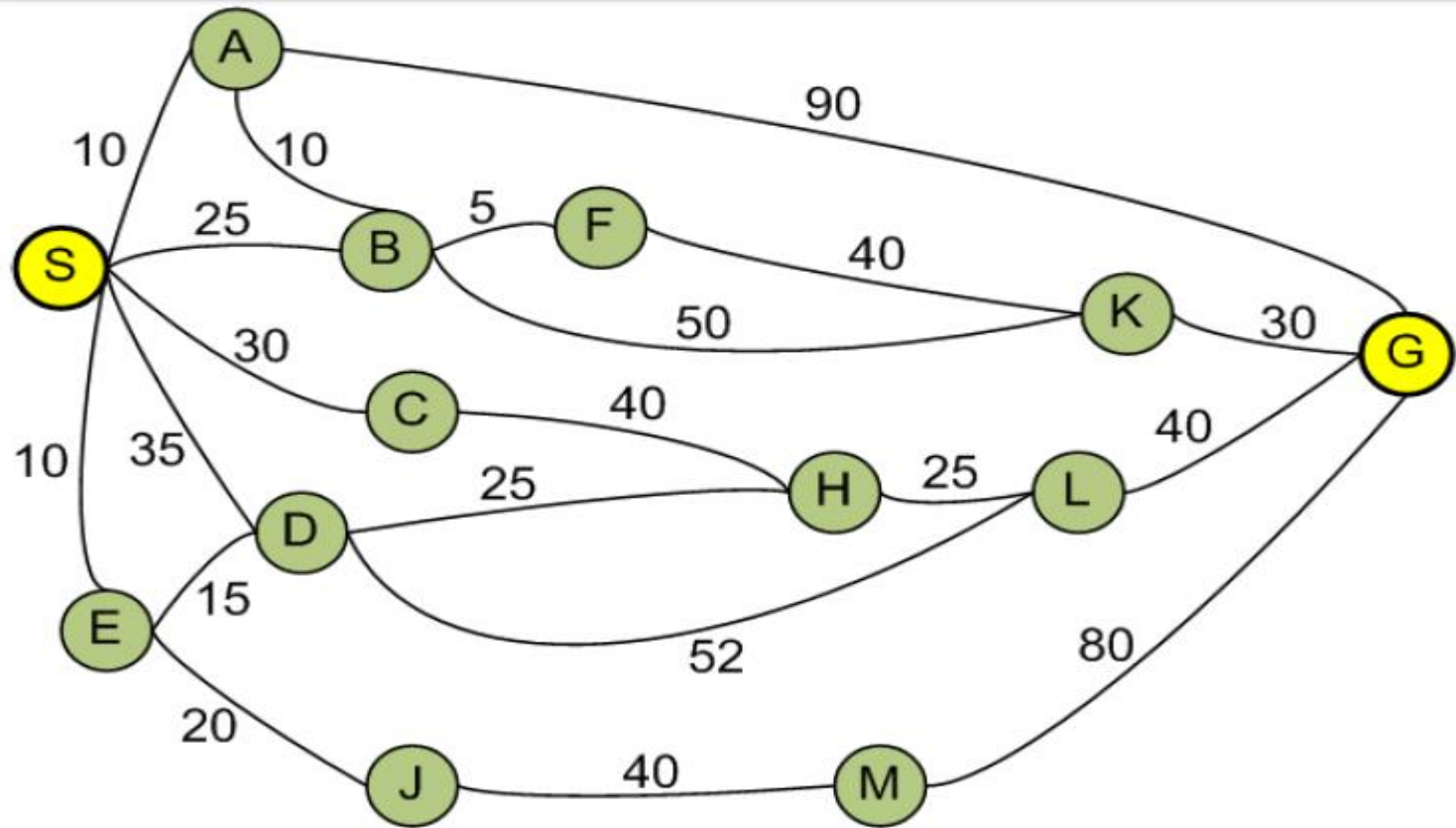
end

 insert *s* in *Queue*

end



Contoh kasus



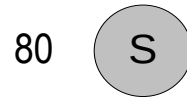
Jarak Garis Lurus Kota n ke Kota G

n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

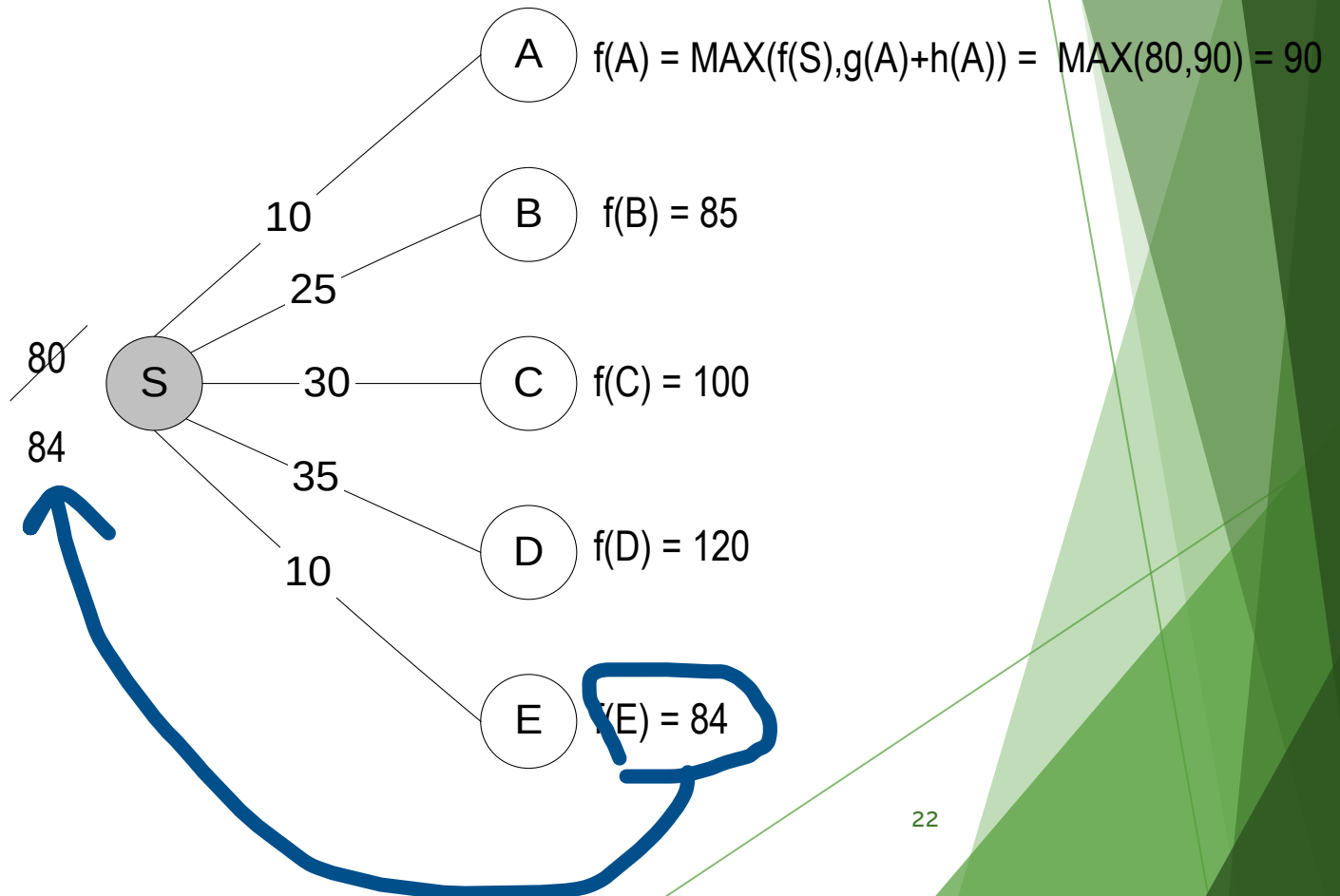
Ketentuan kasus :

- ▶ Pada kasus ini, misalkan memori komputer hanya mampu menyimpan 6 simpul.
- ▶ Oleh karena itu, level maksimum yang dapat dijangkau oleh SMA* adalah level 5.

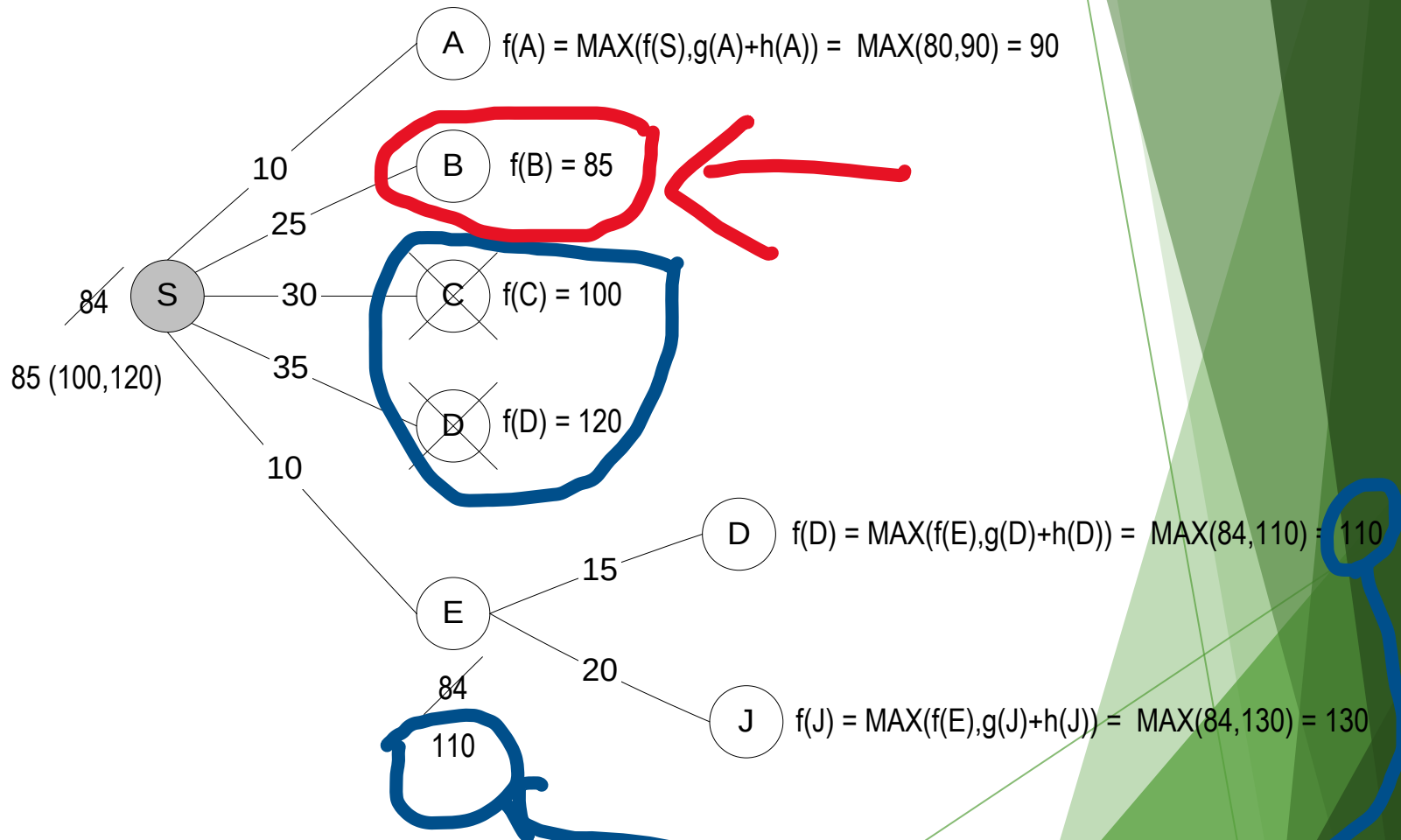
Langkah 1



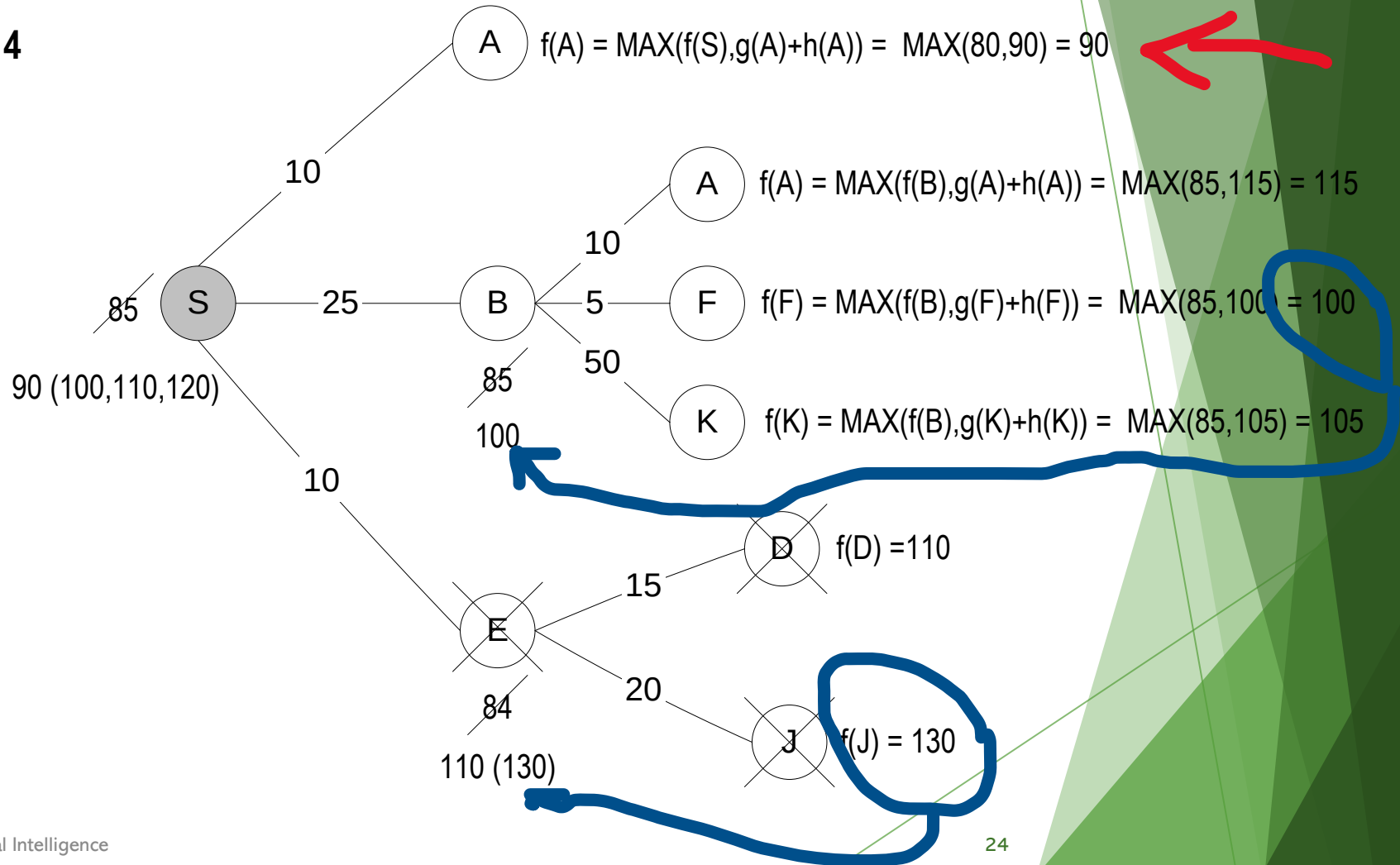
Langkah 2



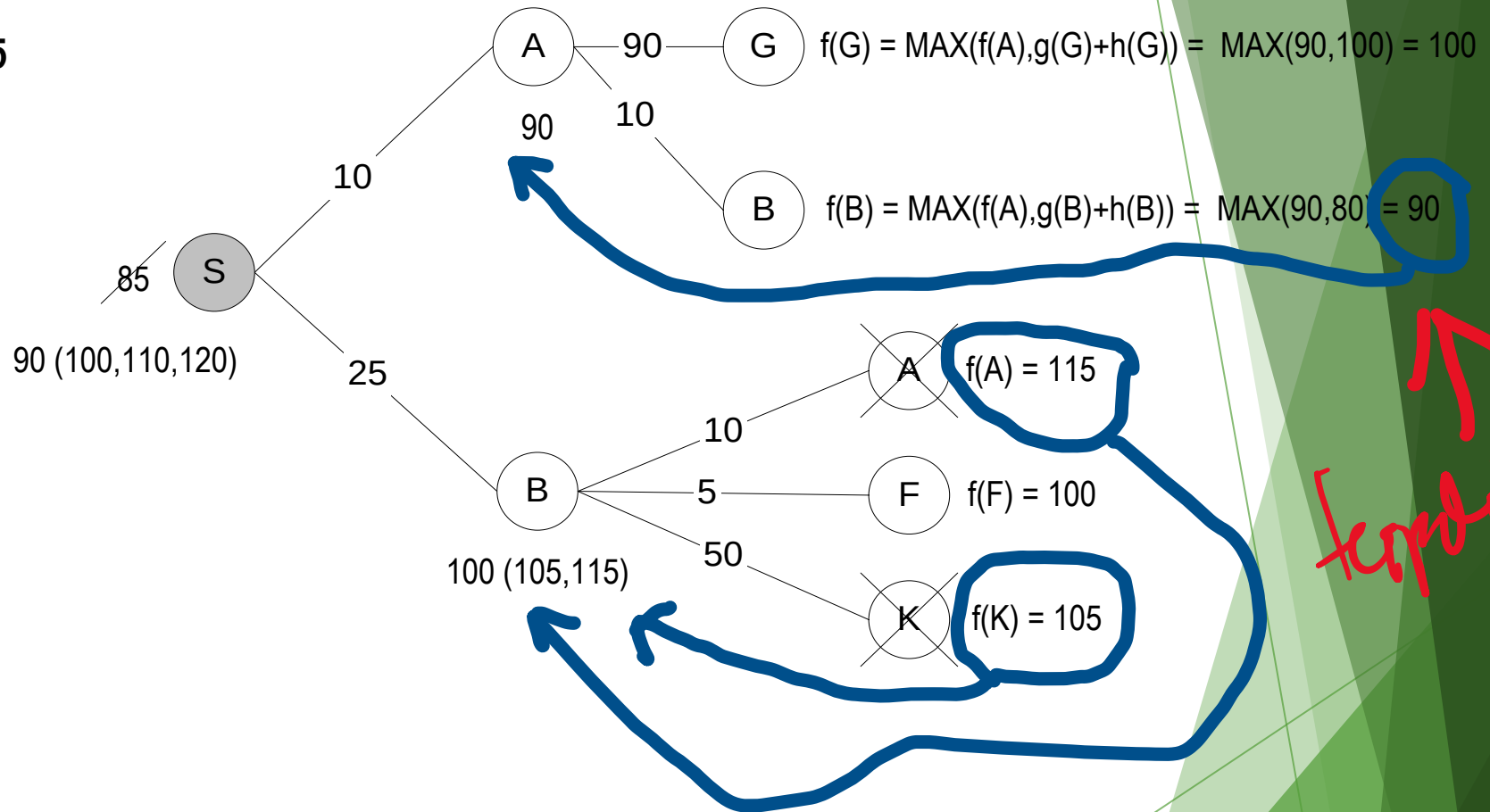
Langkah 3



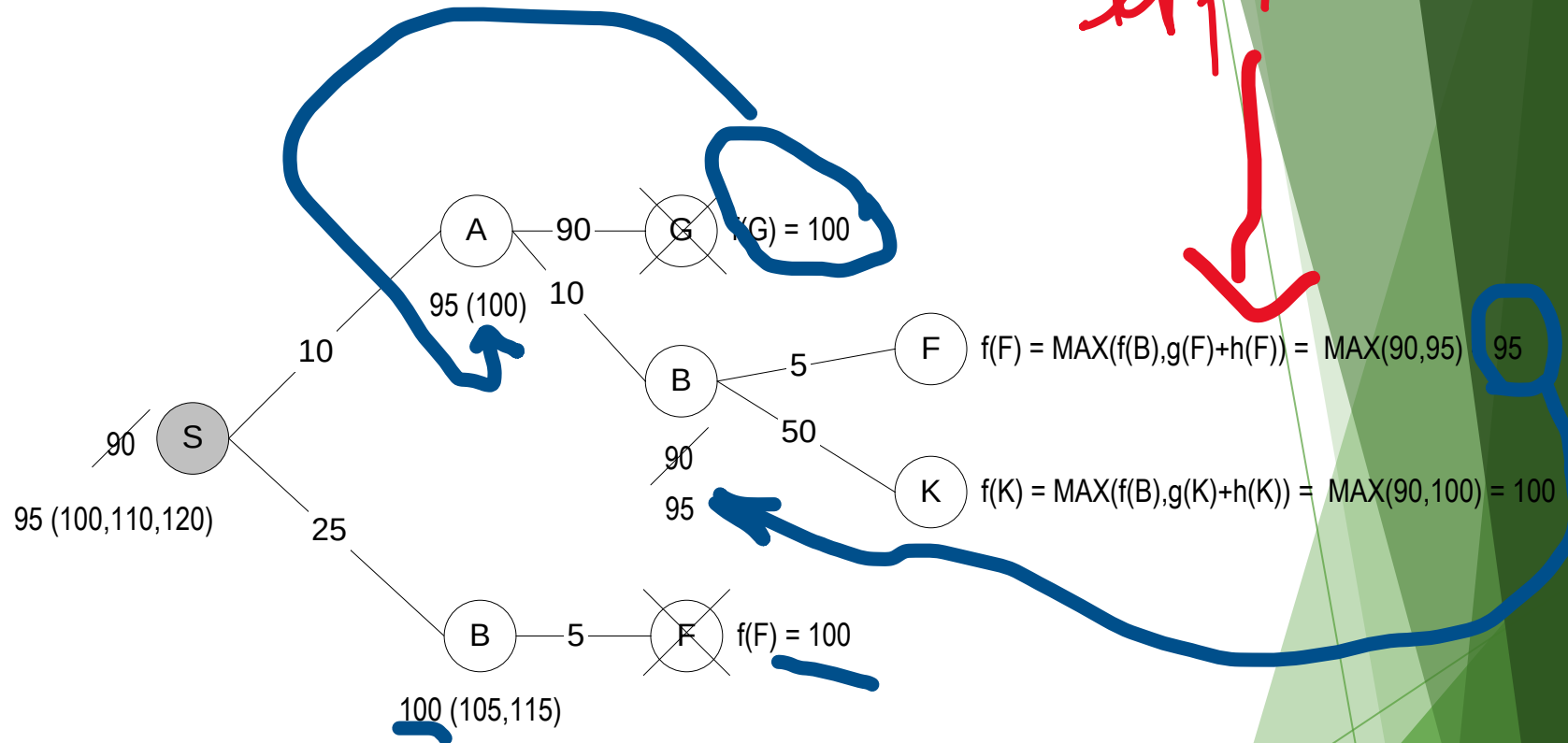
Langkah 4



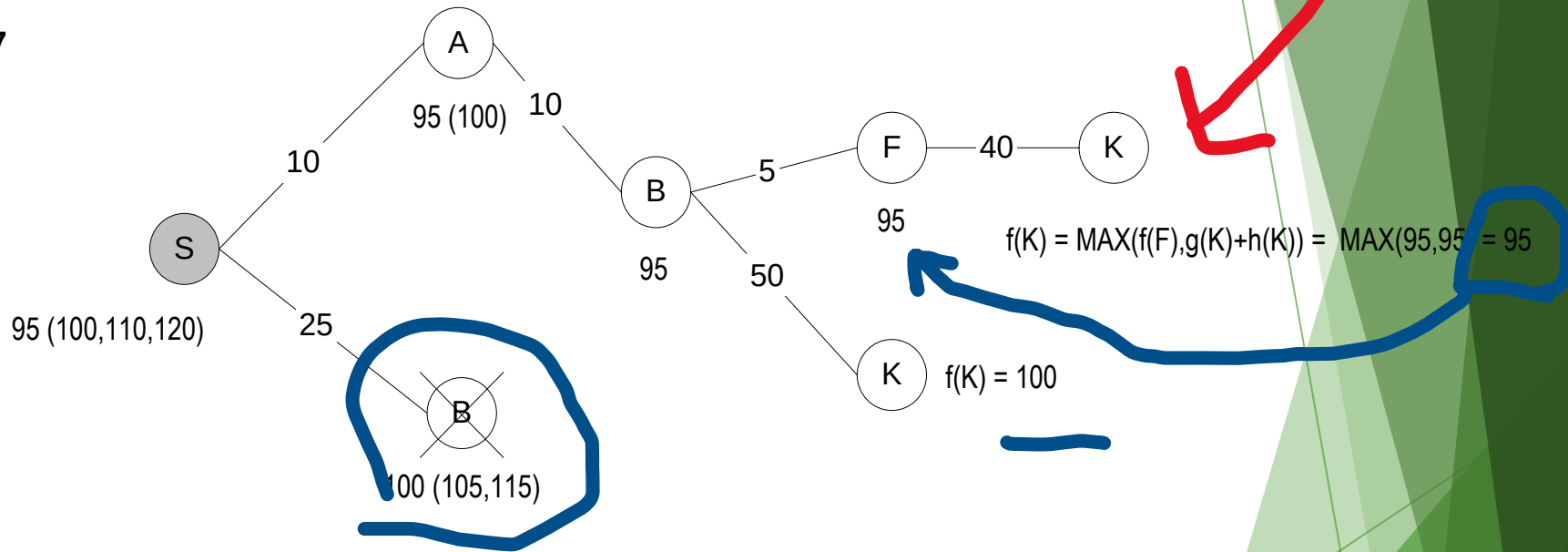
Langkah 5



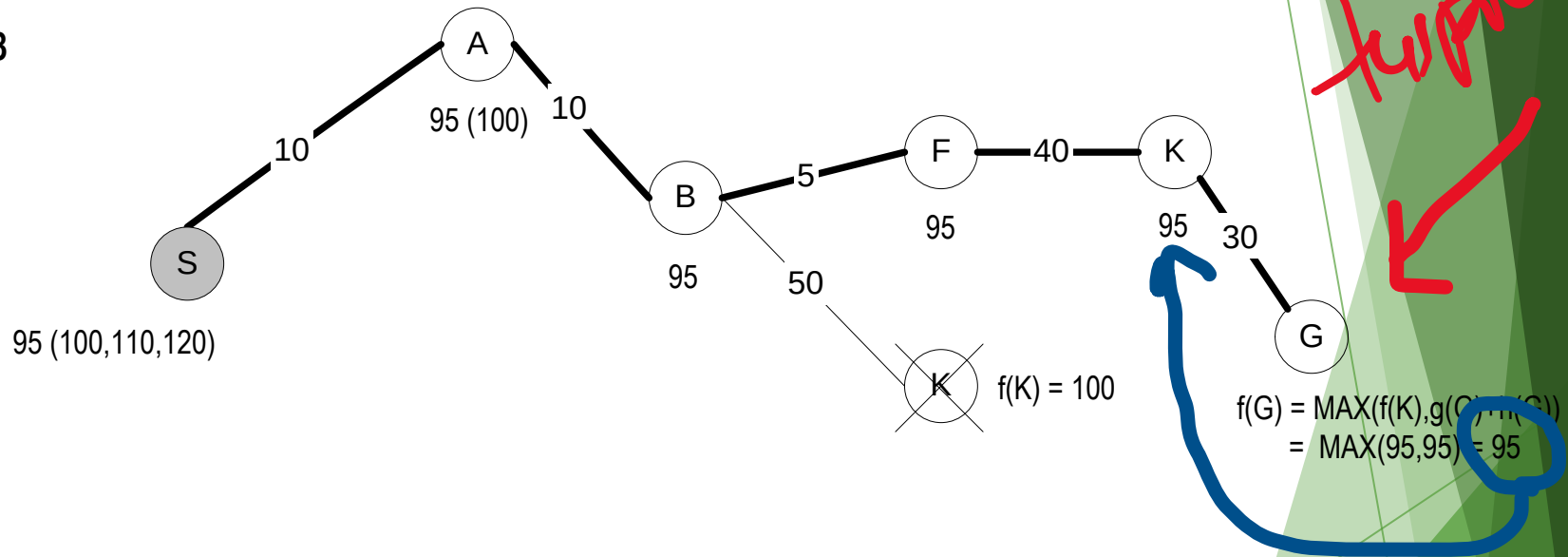
Langkah 6



Langkah 7



Langkah 8



S-A-B-F-K-G

Bi-directional A (BDA*)*

- ▶ Algoritma A^* dari dua arah: simpul asal dan tujuan.
- ▶ Pencarian dihentikan jika *BestNode* dari simpul asal telah berada di dalam *CLOSED* dari simpul tujuan. Cek apakah harus mengganti *parent* dari *BestNode* tersebut dari arah simpul tujuan.
- ▶ Atau sebaliknya, pencarian dihentikan jika *BestNode* dari simpul tujuan telah berada di dalam *CLOSED* dari simpul asal. Cek apakah harus mengganti *parent* dari *BestNode* tersebut dari arah simpul asal.

function BDA*(*masalah*) **returns** *solusi*

inputs: *masalah*: berisi ruang masalah, jenis solusi, S, dan G

local variables: *BNs* dan *BNg*: *BestNode* dari arah start dan *goal*

OPENs dan *OPENg*: *OPEN* dari arah start dan *goal*

CLOSEDs dan *CLOSEDg*: *CLOSED* dari arah start dan *goal*

loop sampai *goal* ditemukan

BNs, *OPENs*, *CLOSEDs* $\leftarrow$ A*(*OPENs*,*CLOSEDs*,S,G)

if *BNs* in *CLOSEDg* **then** { *goal* sudah ditemukan }

$v \leftarrow$ simpul di *CLOSEDg* yang sama dengan *BNs*

g-terbaik $\leftarrow g(G,v)$ { biaya sebenarnya dari G ke *v* melalui *parent* lama }

loop untuk semua *suk* = simpul di *CLOSEDg* yang merupakan suksesor *v*

if *g*(G,*v*) melalui *suk* < *g*-terbaik **then**

g -terbaik $\leftarrow g(G,v)$ melalui *suk*

parent-baru $\leftarrow$ *suk*

end

end

g(*v*) $\leftarrow$ *g*-terbaik

parent dari *v* $\leftarrow$ *parent-baru*

returns *solusi*

end

BNg, *OPENg*, *CLOSEDg* $\leftarrow$ A*(*OPENg*,*CLOSEDg*,G,S)

if *BNg* in *CLOSEDs* **then** { *goal* sudah ditemukan }

$u \leftarrow$ simpul di *CLOSEDs* yang sama dengan *BNg*

g-terbaik $\leftarrow g(S,u)$ { biaya sebenarnya dari S ke *u* melalui *parent* lama }

loop untuk semua *suk* = simpul di *CLOSEDg* yang merupakan suksesor *u*

if *g*(S,*u*) melalui *suk* < *g*-terbaik **then**

g -terbaik $\leftarrow g(S,u)$ melalui *suk*

parent-baru $\leftarrow$ *suk*

end

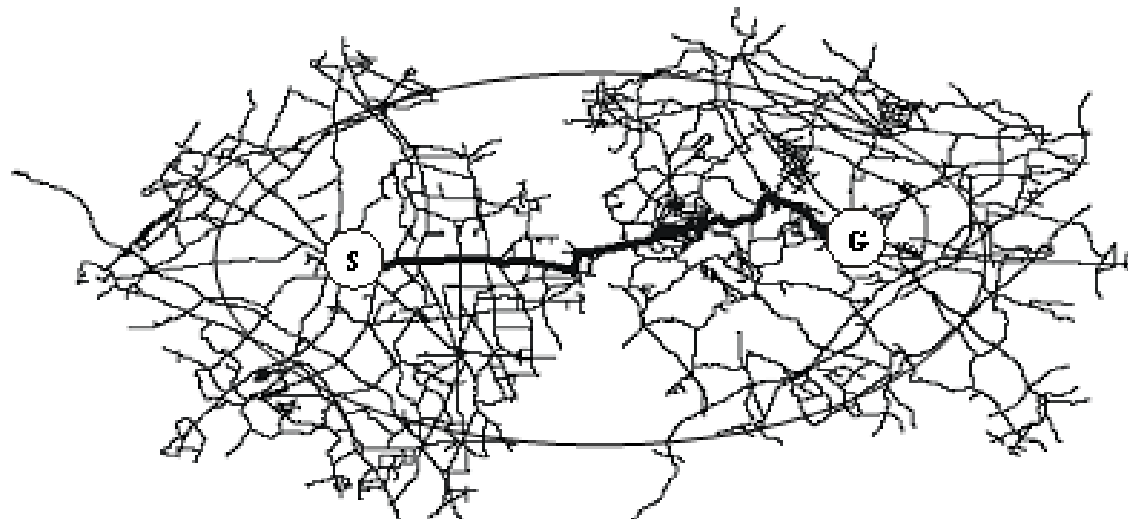
end

g(S,*u*) $\leftarrow$ *g*-terbaik

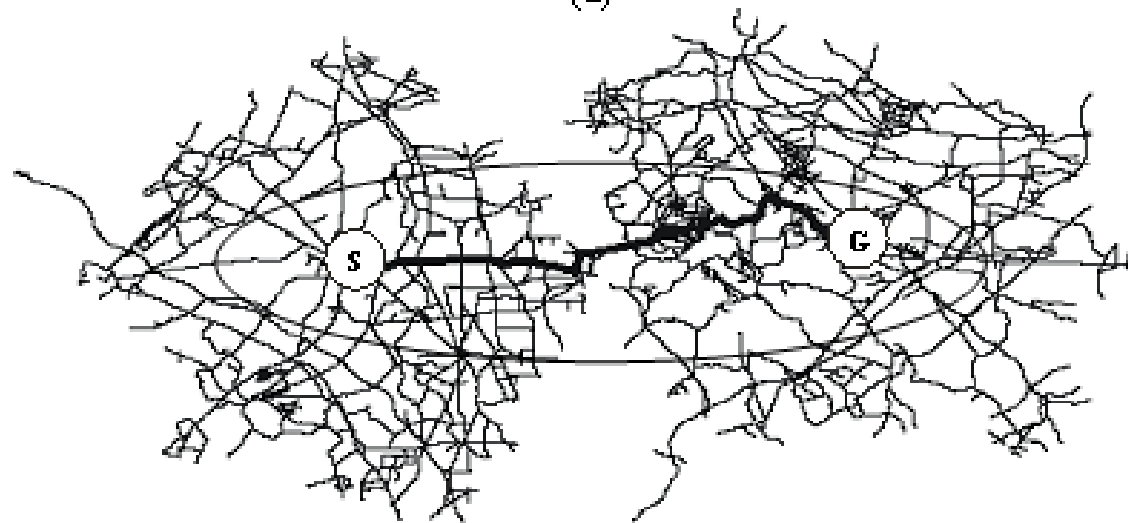
parent dari *u* $\leftarrow$ *parent-baru*

returns *solusi*

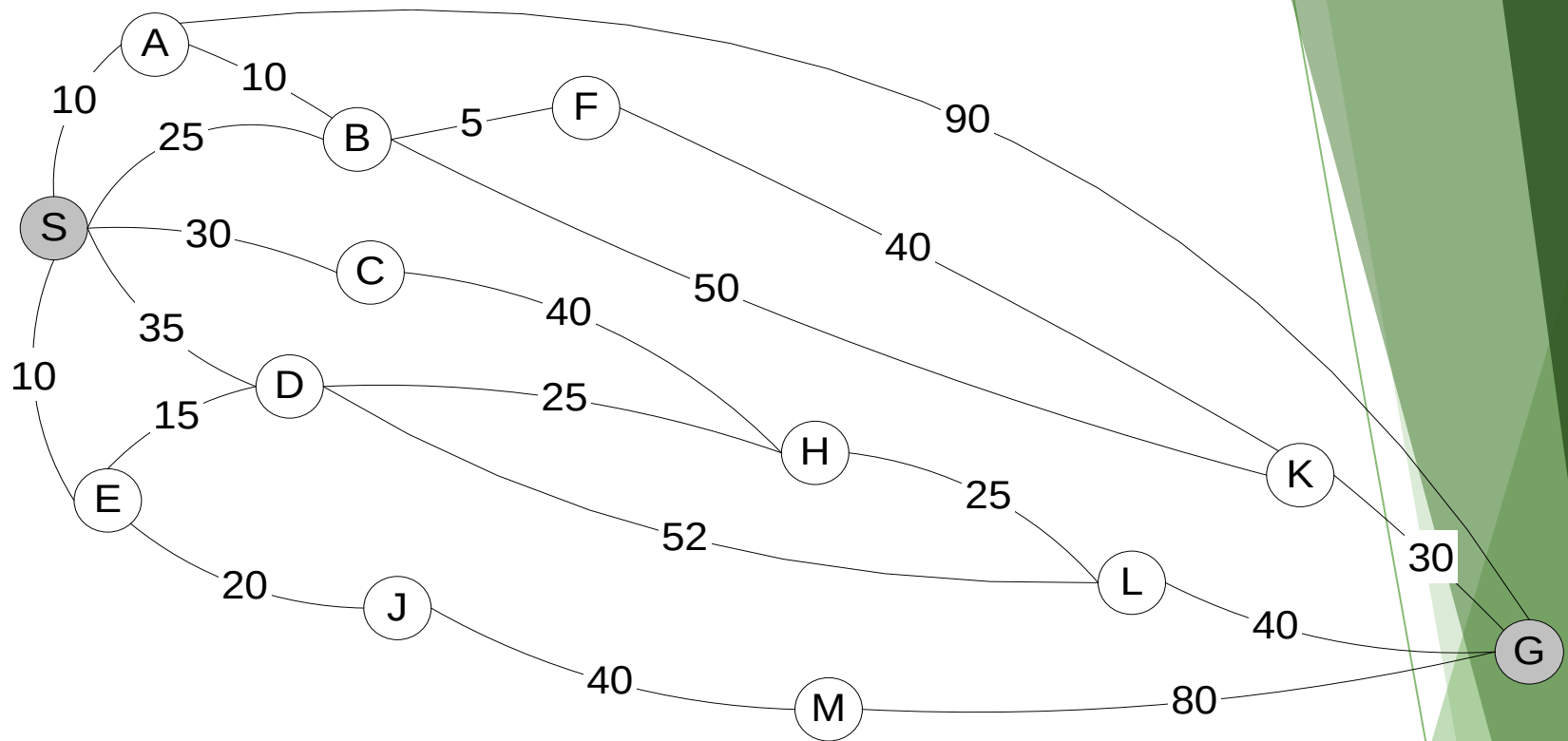
end



(a)



(b)

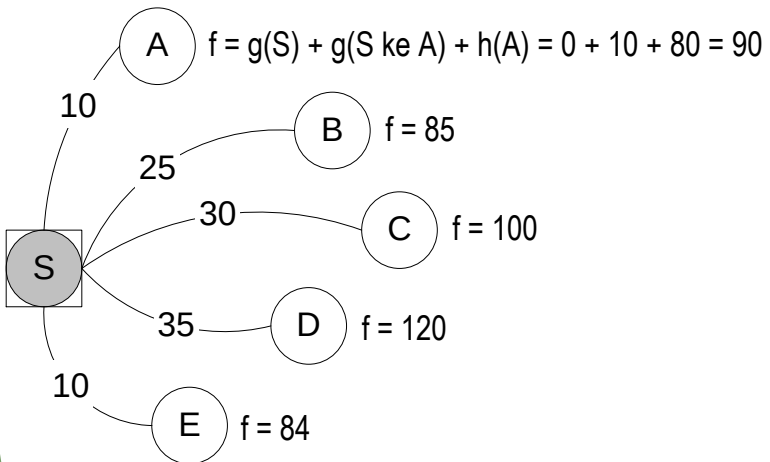


n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h_s(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h_g(n)$	0	10	15	25	30	5	20	90	45	25	50	70	60

Pencarian Maju (dari S ke G)

Langkah 1

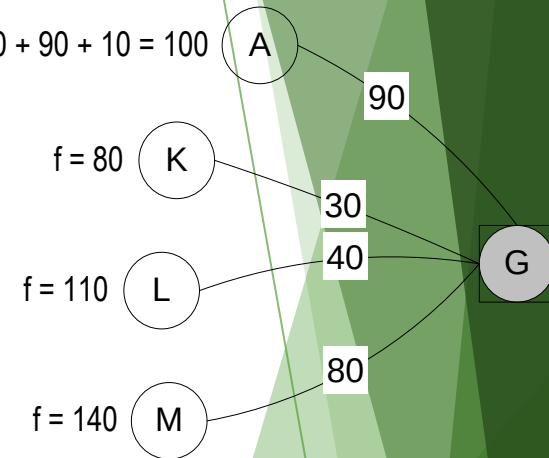


OPEN=[A, B, C, D, E]

CLOSED=[S]

Pencarian Mundur (dari G ke S)

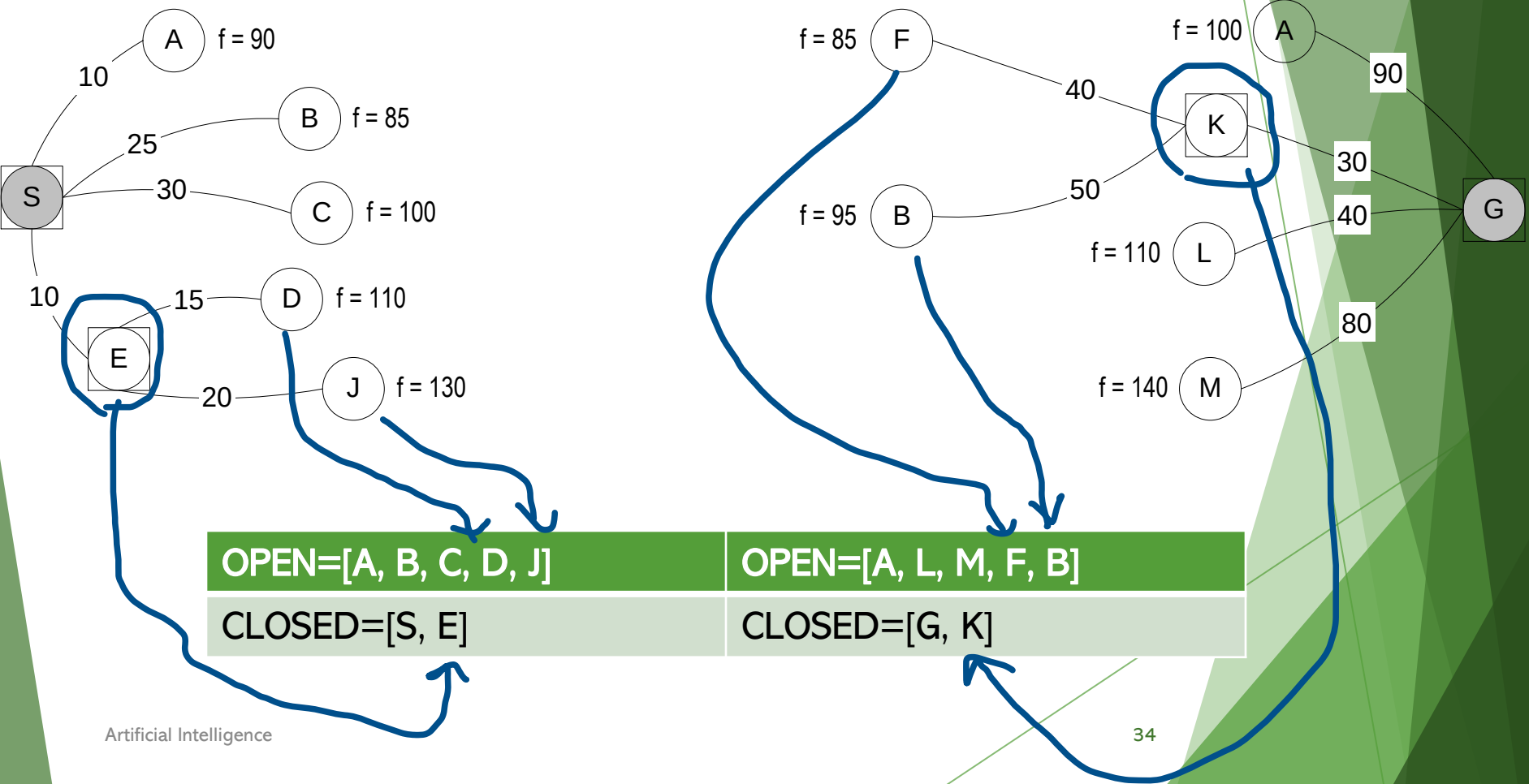
$$f = g(G) + g(G \text{ ke } A) + h'(A) = 0 + 90 + 10 = 100$$



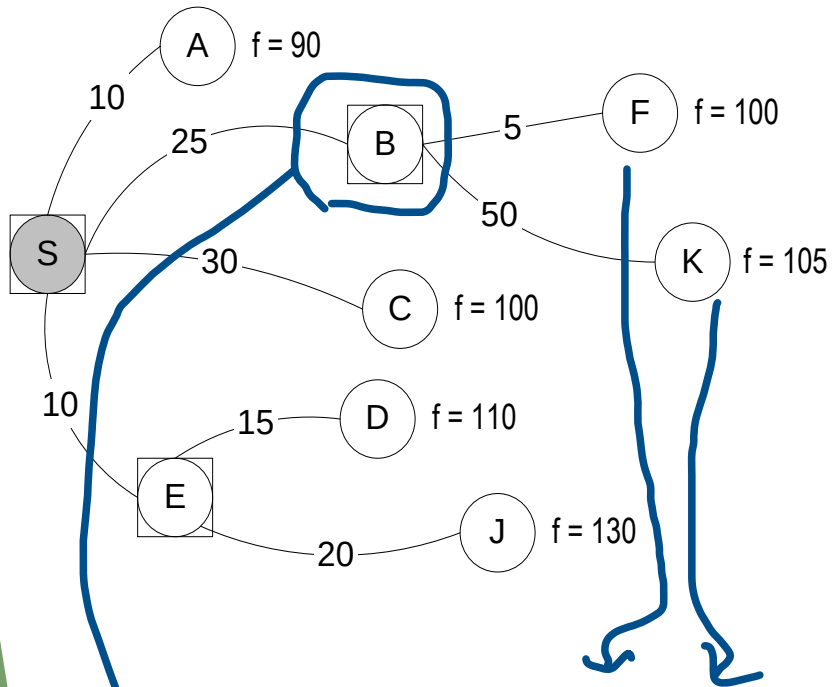
OPEN=[A, K, L, M]

CLOSED=[G]

Langkah 2

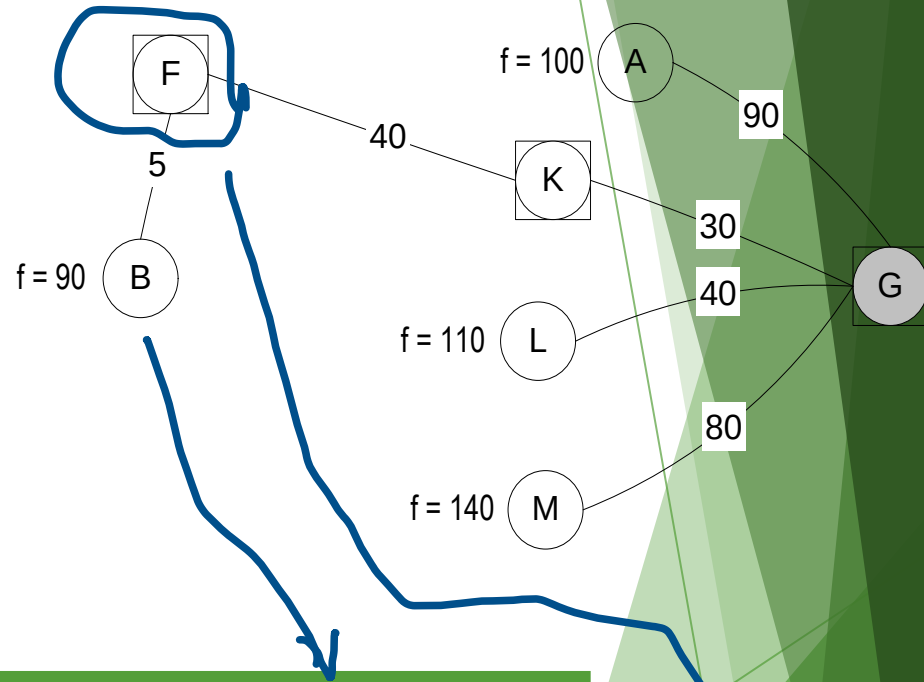


Langkah 3



OPEN=[A, C, D, F, J, K]

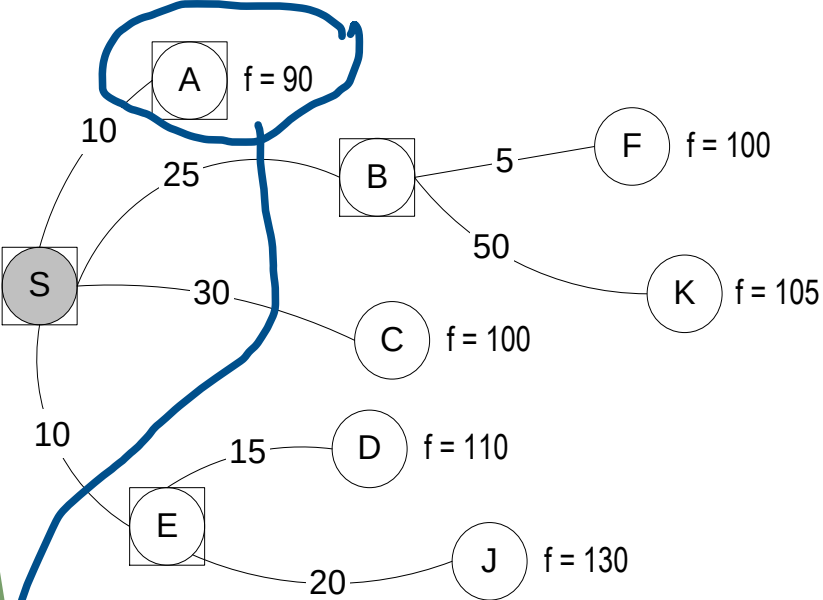
CLOSED=[S, E, B]



OPEN=[A, L, M, B]

CLOSED=[G, K, F]

Langkah 4

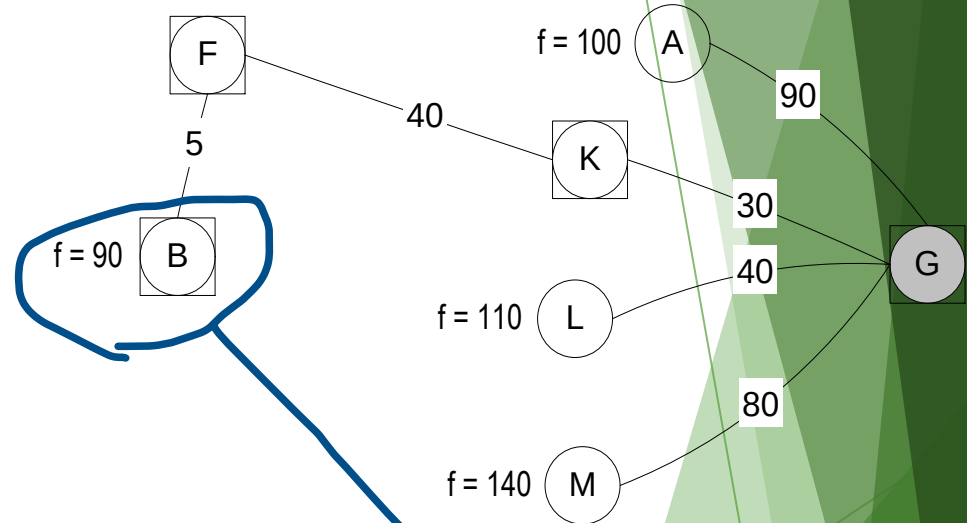


OPEN=[C, D, F, J, K, G]

CLOSED=[S, E, B, A]

OPEN=[A, L, M]

CLOSED=[G, K, F, B]



- ▶ Pada Langkah ke-4 dalam arah mundur, simpul B terpilih, Tetapi simpul tersebut telah berada di CLOSED arah maju → jadi solusi sudah ditemukan.

OPEN=[A, C, D, F, J, K]	OPEN=[A, L, M, B]
CLOSED=[S, E, B, A]	CLOSED=[G, K, F]

- ▶ Cari **suksesor B** yang berada di CLOSED arah maju, yaitu **A**
- ▶ Perbaiki f, dimana $g(S, B)$ melalui A < $g(S, B)$ → parent B diubah yang tadinya S menjadi A.
- ▶ Selanjutnya rute dan total biaya di telusuri balik dari G-S dan S-G, hasilnya digabungkan, sehingga menjadi solusi utuh yaitu : S-A-B-F-K-G, dengan total jarak 95 km.
- ▶ Algoritma BD* → **optimal** dan **complete**

Modified Bi-directional A* (MBDA*)

- Fungsi heuristik untuk simpul n pada pencarian maju (dari S ke G):

$$f = g(S, n) + \frac{1}{2} [h_s(n) - h_g(n)]$$

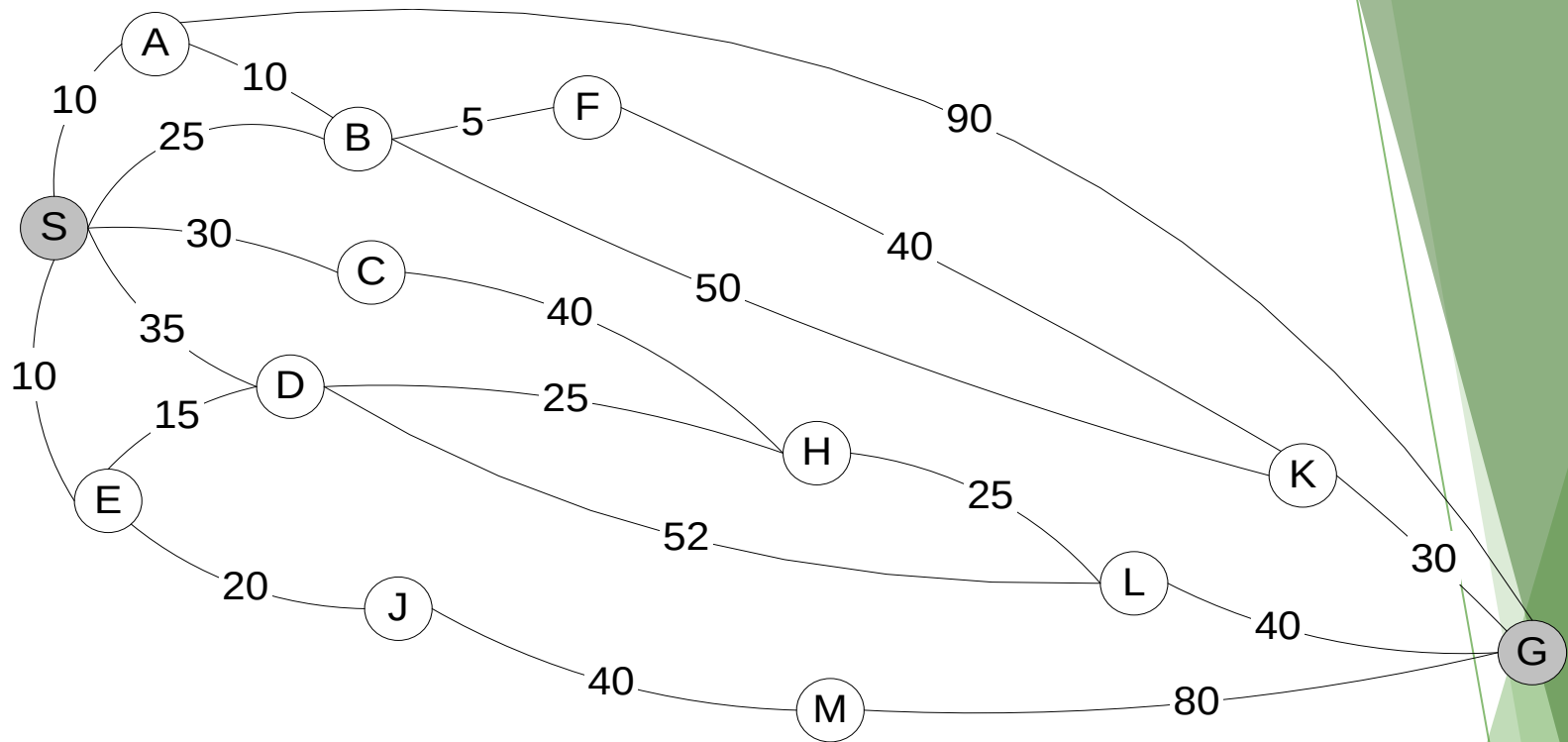
- Fungsi heuristik untuk simpul n pada pencarian mundur (dari G ke S):

$$f = g(G, n) + \frac{1}{2} [h_g(n) - h_s(n)]$$

Modified Bi-directional A* (MBDA*)

- ▶ S : simpul asal atau *initial state*
- ▶ G : simpul tujuan atau *goal state*
- ▶ $g(S,n)$: biaya sebenarnya dari S ke n
- ▶ $g(G,n)$: biaya sebenarnya dari G ke n
- ▶ $h_s(n)$: biaya perkiraan dari n ke G
- ▶ $h_g(n)$: biaya perkiraan dari n ke S

- ▶ MBDA* adalah algoritma A* yang dilakukan dalam dua arah secara serentak, yaitu dari arah simpul awal ke simpul tujuan dan dari arah simpul tujuan ke simpul awal.
- ▶ Proses pencarian akan berhenti jika **Best Node** dari arah simpul asal telah berada dalam senarai CLOSED dari arah simpul tujuan, yang kemudian dilakukan pemeriksaan apakah perlu dilakukan pergantian parent terhadap **Best Node** tersebut.
- ▶ Pencarian juga berhenti jika sebaliknya, yaitu jika **Best Node** dari arah simpul tujuan telah berada dalam senarai CLOSED dari arah simpul awal.
- ▶ Dibandingkan dengan algoritma A*, untuk masalah yang besar MBDA* bisa menyelesaikan dengan cepat dan hemat dalam pemakaian memori karena MBDA* membangkitkan jumlah simpul yang lebih sedikit dibandingkan A*

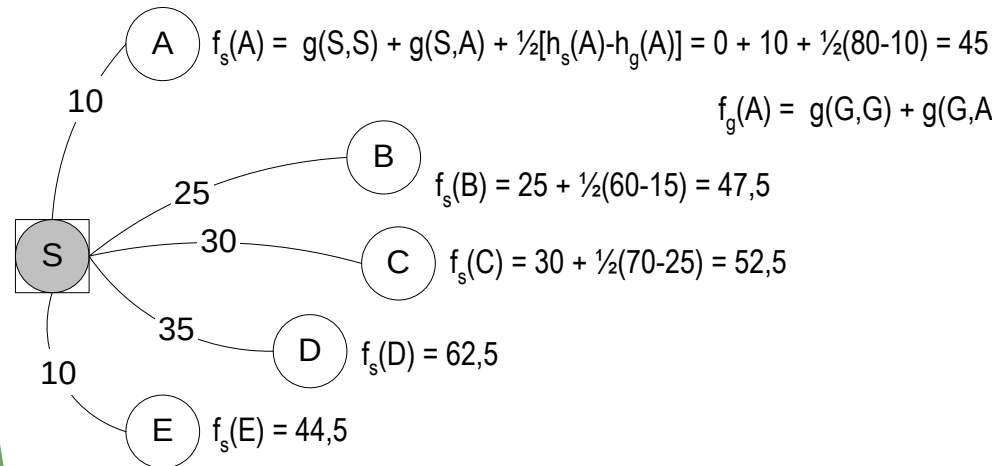


n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h_s(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

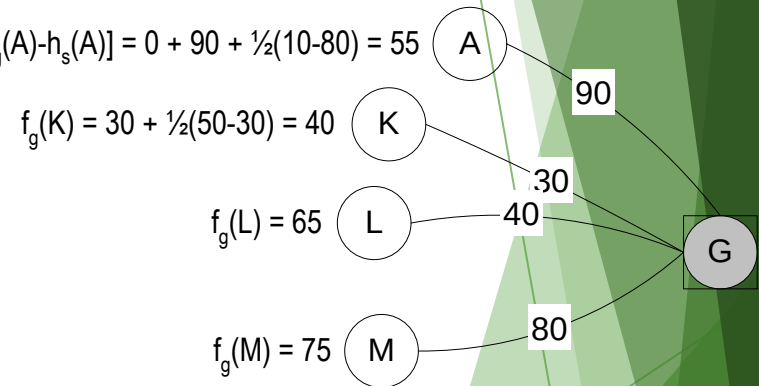
n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h_g(n)$	0	10	15	25	30	5	20	90	45	25	50	70	60

Pencarian Maju (dari S ke G)

Langkah 1



Pencarian Mundur (dari G ke S)



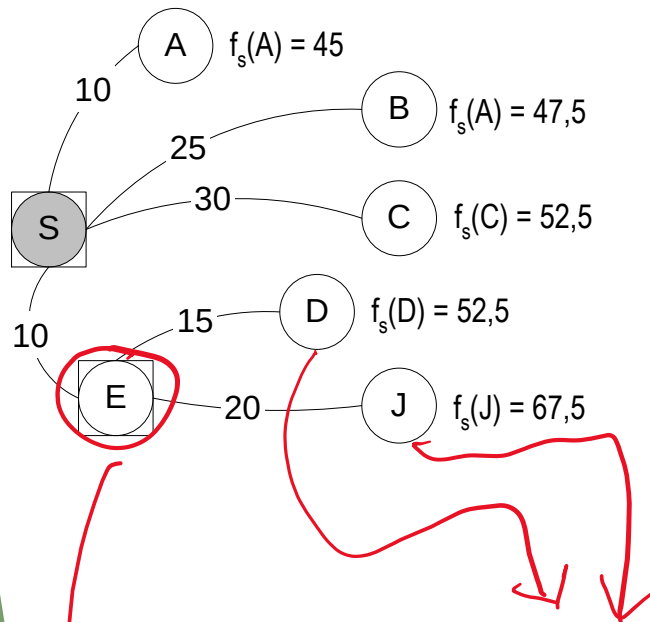
OPEN=[A, B, C, D, E]

CLOSED=[S]

OPEN=[A, K, L, M]

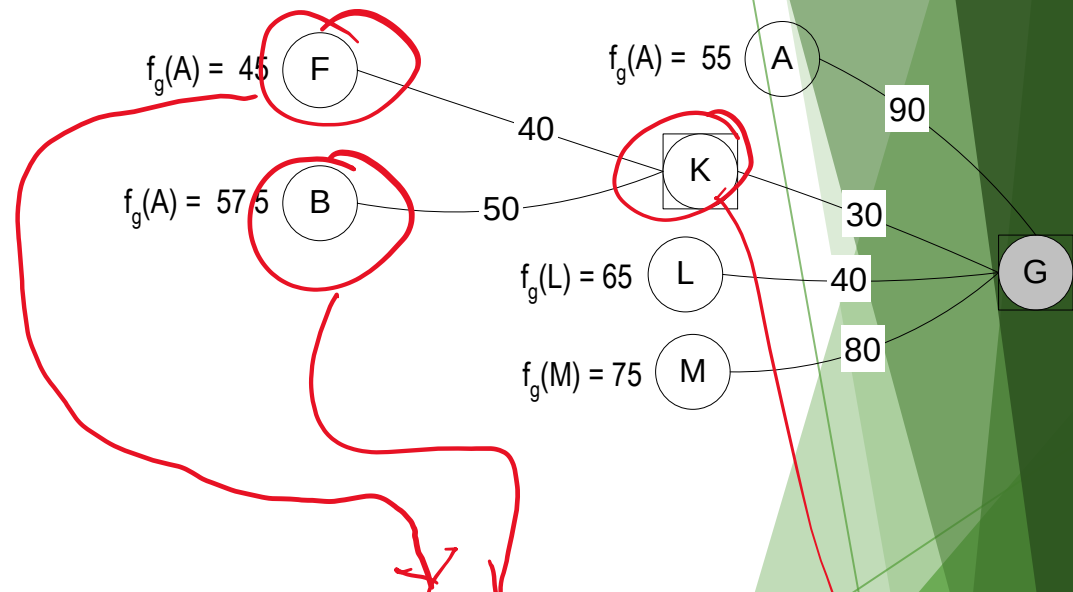
CLOSED=[G]

Langkah 2



OPEN=[A, B, C, D, J]

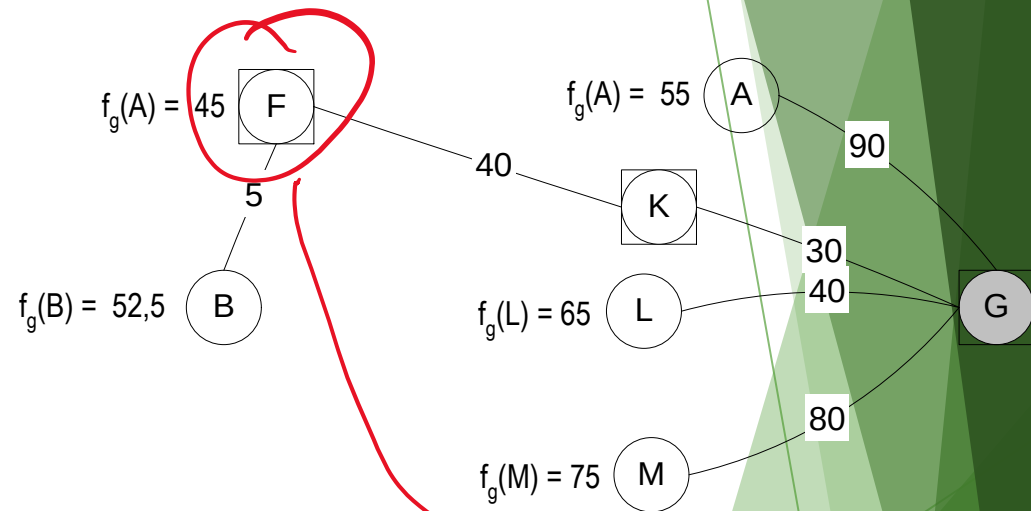
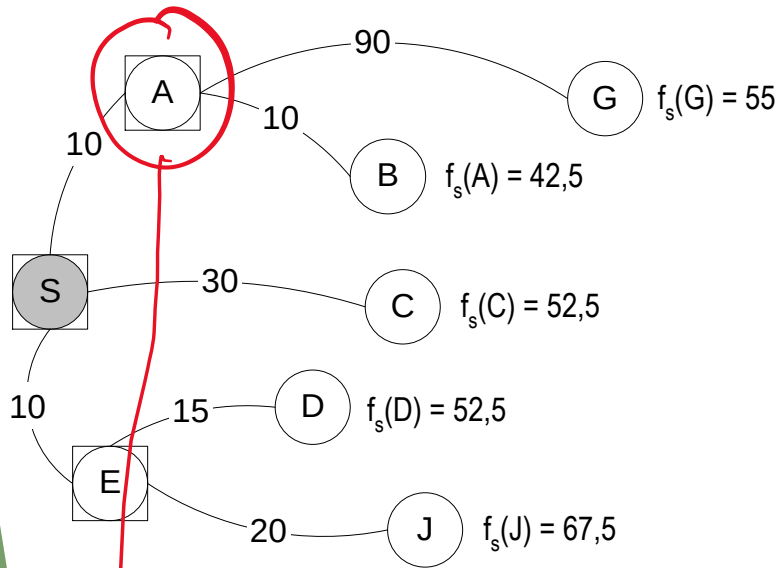
CLOSED=[S, E]



OPEN=[A, L, M, F, B]

CLOSED=[G, K]

Langkah 3



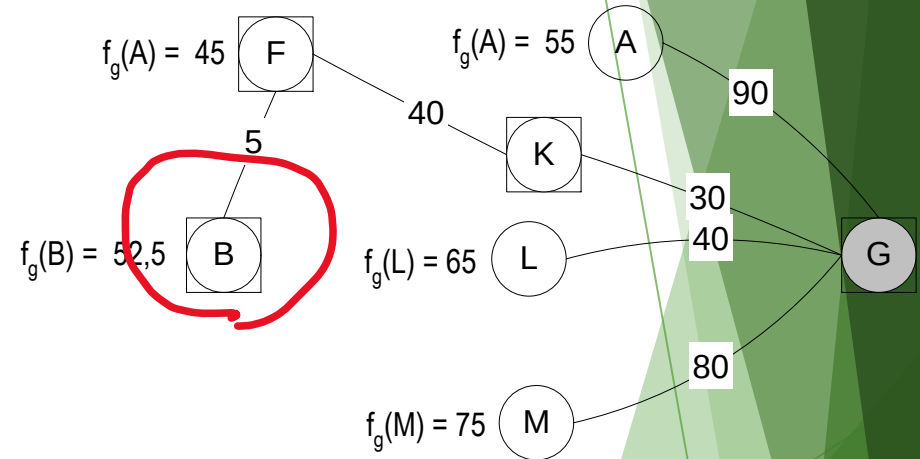
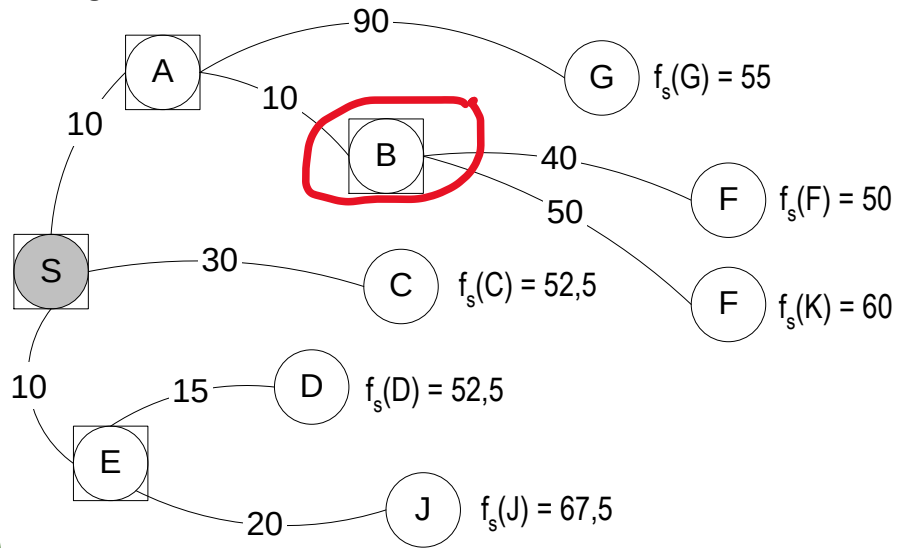
OPEN=[B, C, D, J, G]

CLOSED=[S, E, A]

OPEN=[A, L, M, B]

CLOSED=[G, K, F]

Langkah 4



OPEN=[C, D, J, G, F, K]

CLOSED=[S, E, A, B]

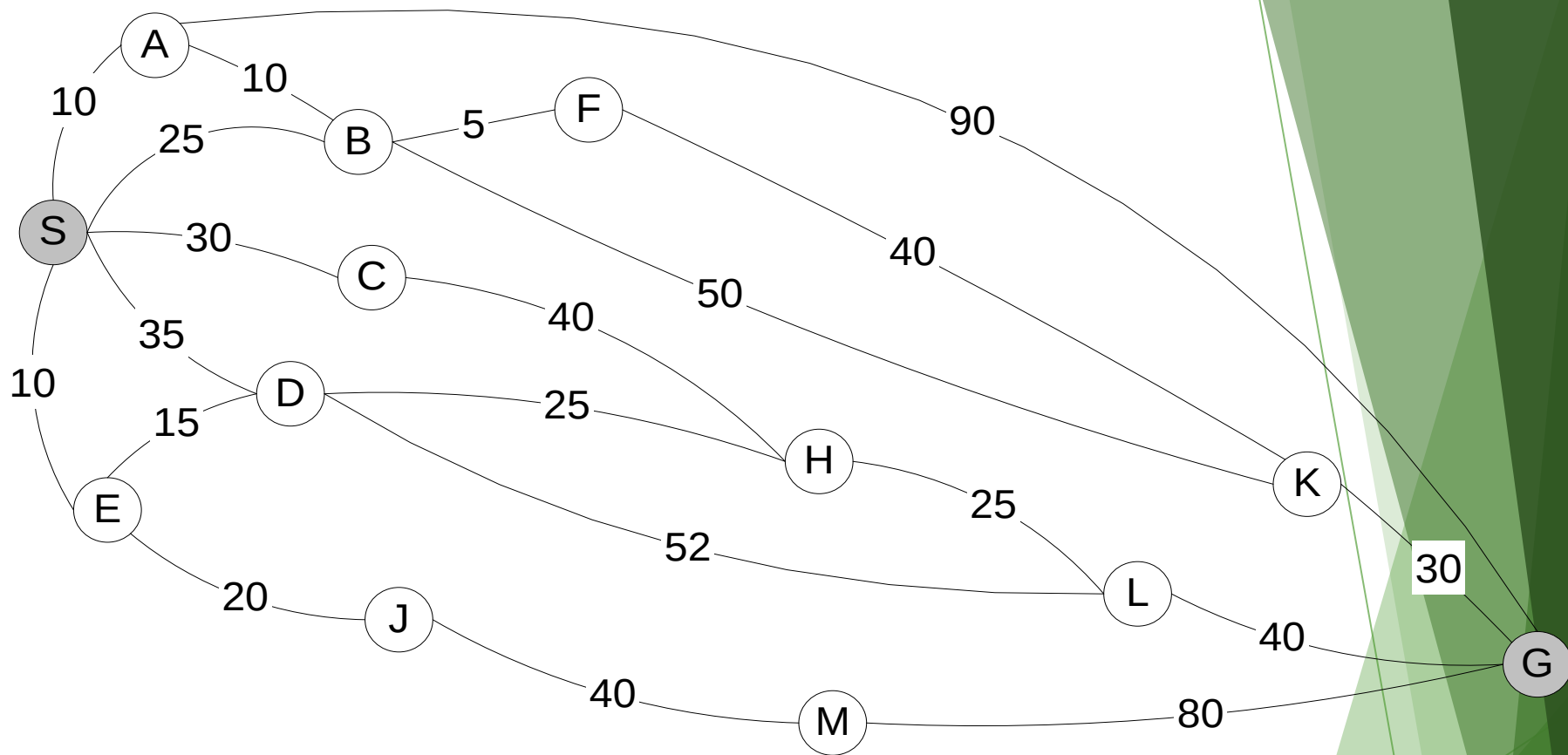
OPEN=[A, L, M, B]

CLOSED=[G, K, F]

Dynamic Weighting A* (DWA*)

- ▶ Fungsi heuristik *h* diberi bobot dinamis.
- ▶ Pada awal iterasi, lebih baik pencarian dilakukan ke arah mana saja.
- ▶ Tetapi, ketika *goal* sudah dekat, barulah pencarian difokuskan ke arah *goal*.
- ▶ Fungsi heuristik yang digunakan

$$f(n) = g(n) + w(n) * h(n)$$

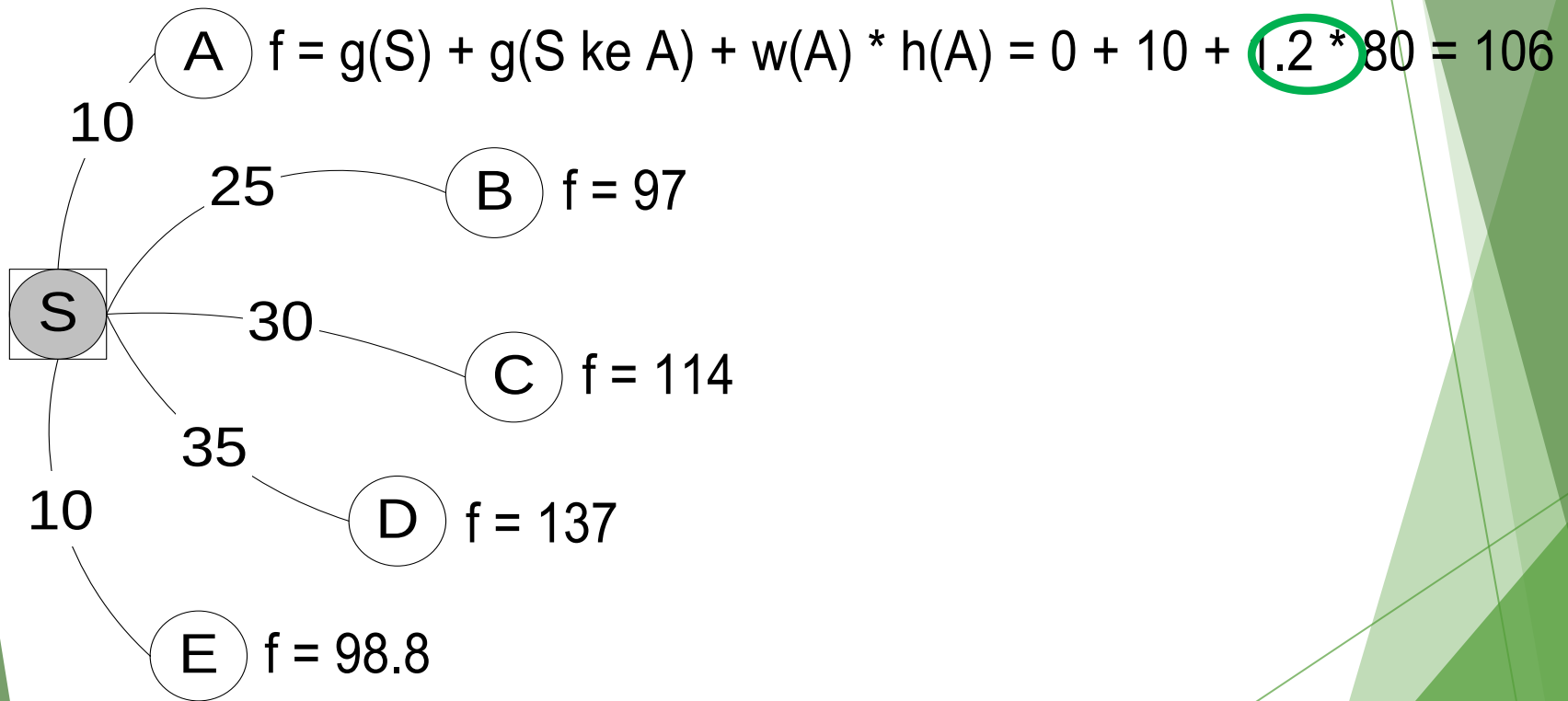


n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

OPEN=[A, B, C, D, E]

CLOSED=[S]

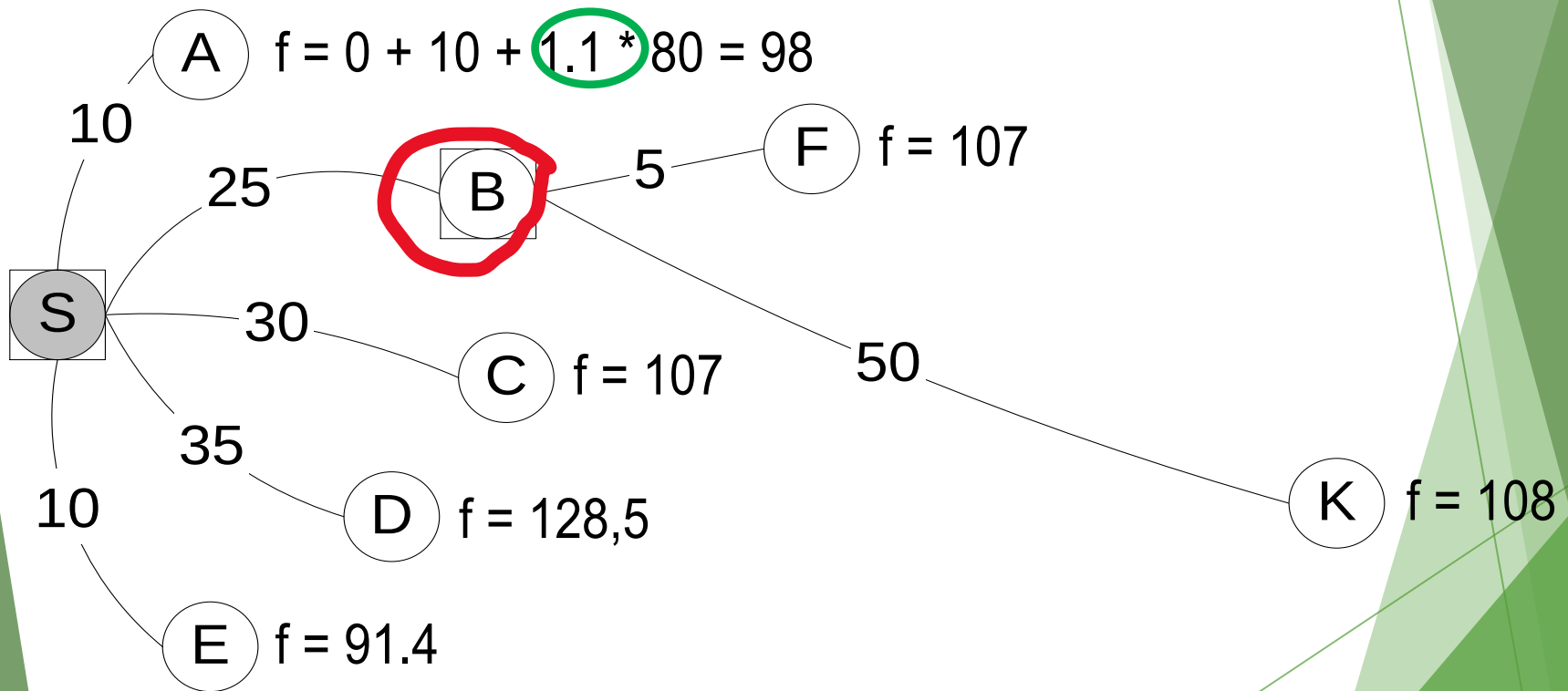
Langkah 1



OPEN=[A, C, D, E, F, K]

CLOSED=[S, B]

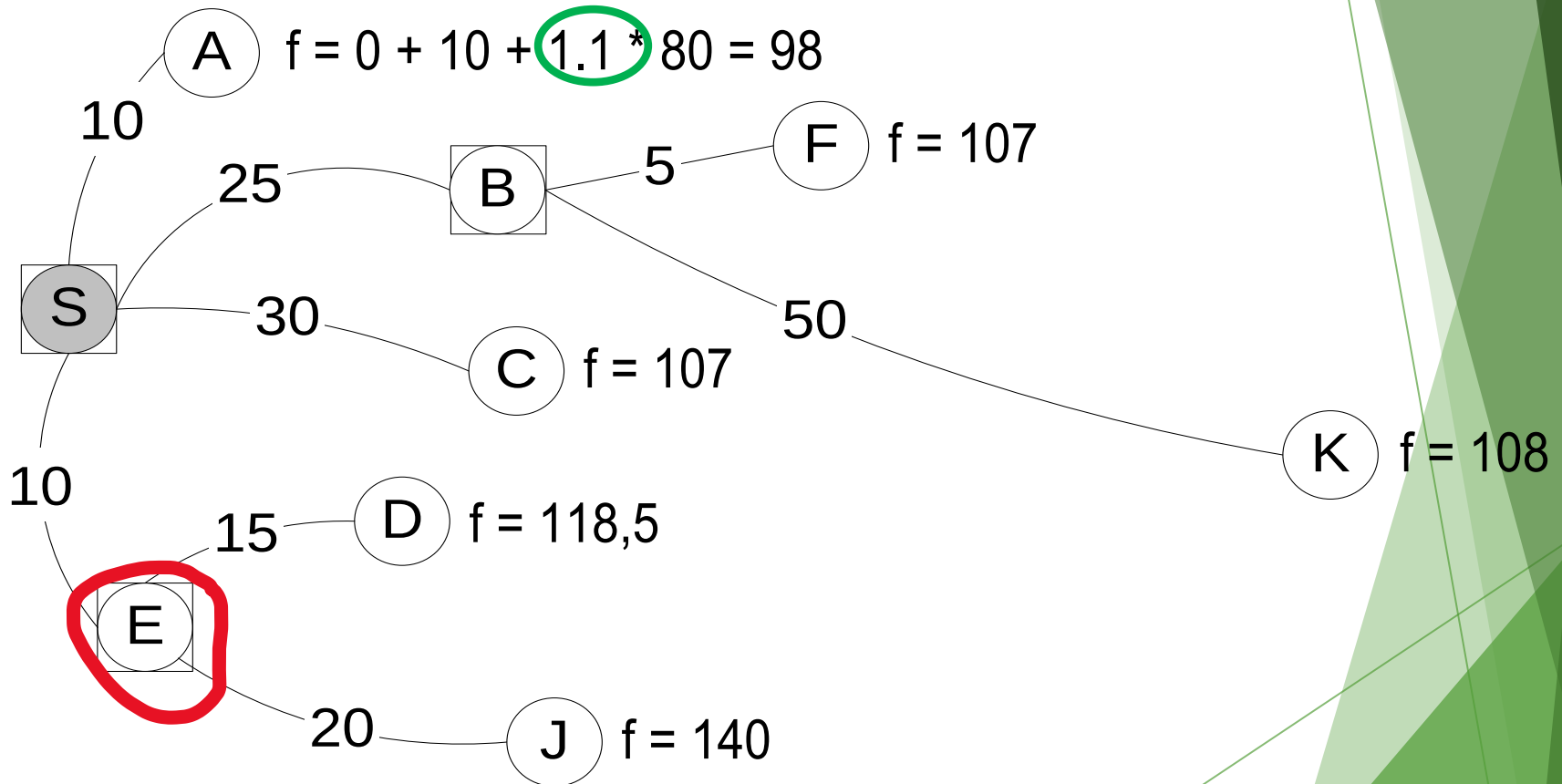
Langkah 2



OPEN=[A, C, D, F, K, J]

CLOSED=[S, B, E]

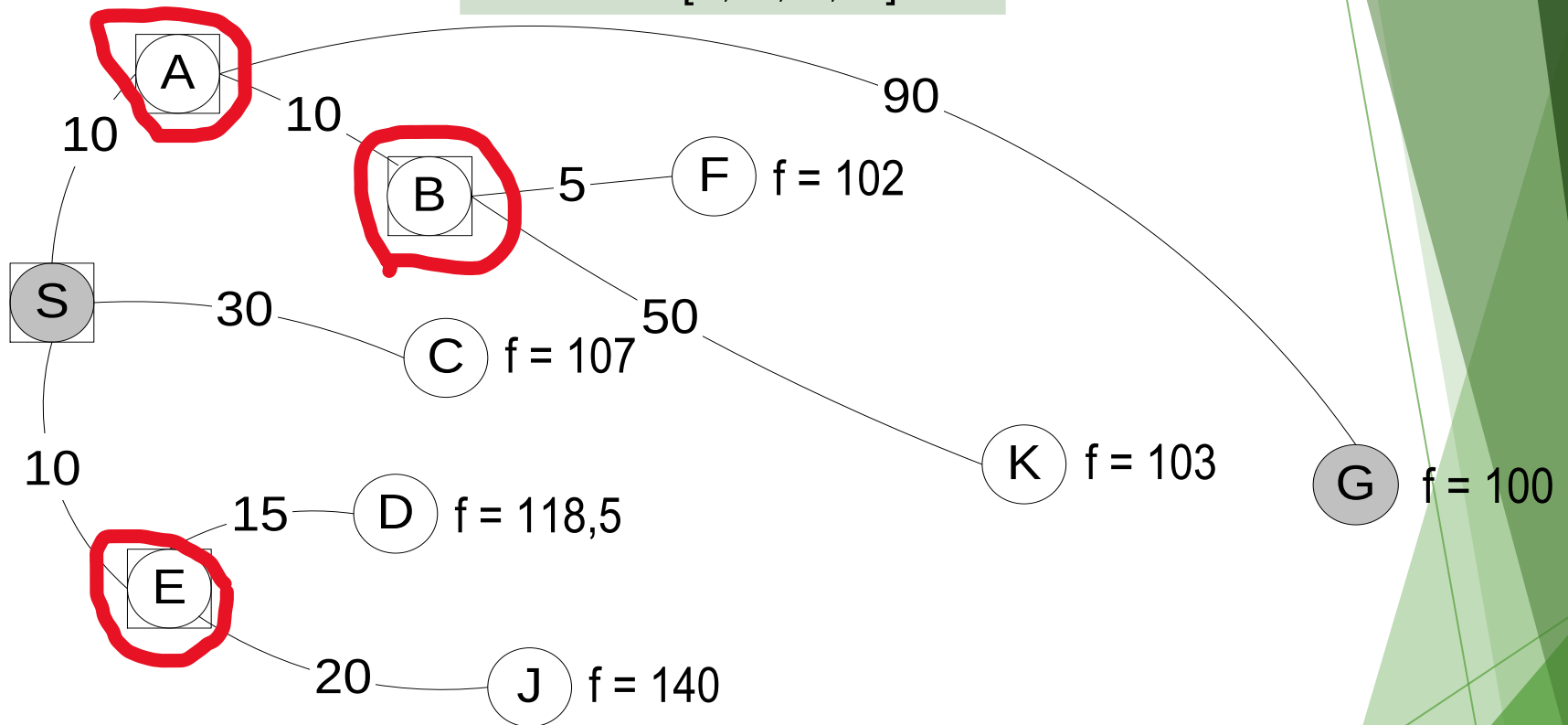
Langkah 3



OPEN=[C, D, F, G, K, J]

CLOSED=[S, B, E, A]

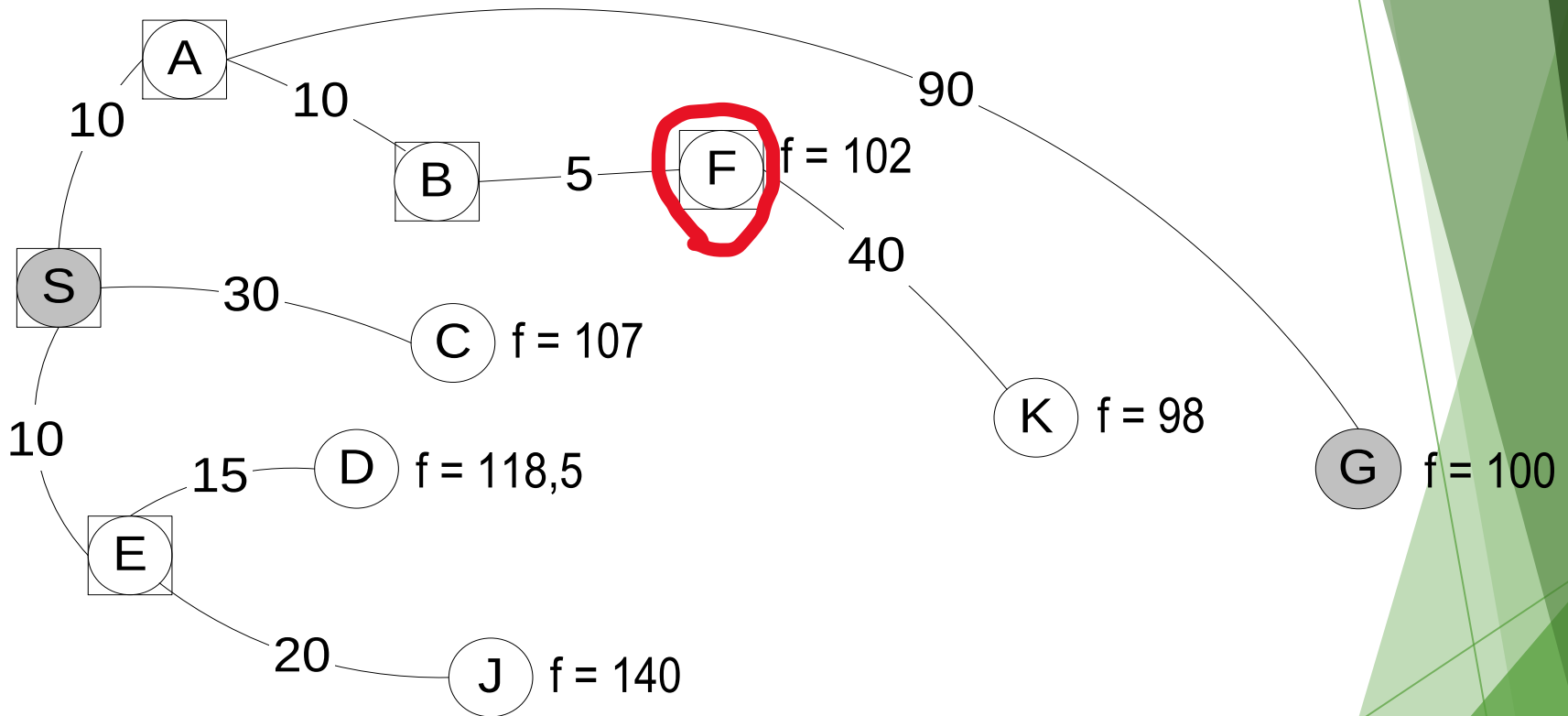
Langkah 4



OPEN=[C, D, G, K, J]

CLOSED=[S, B, E, A, F]

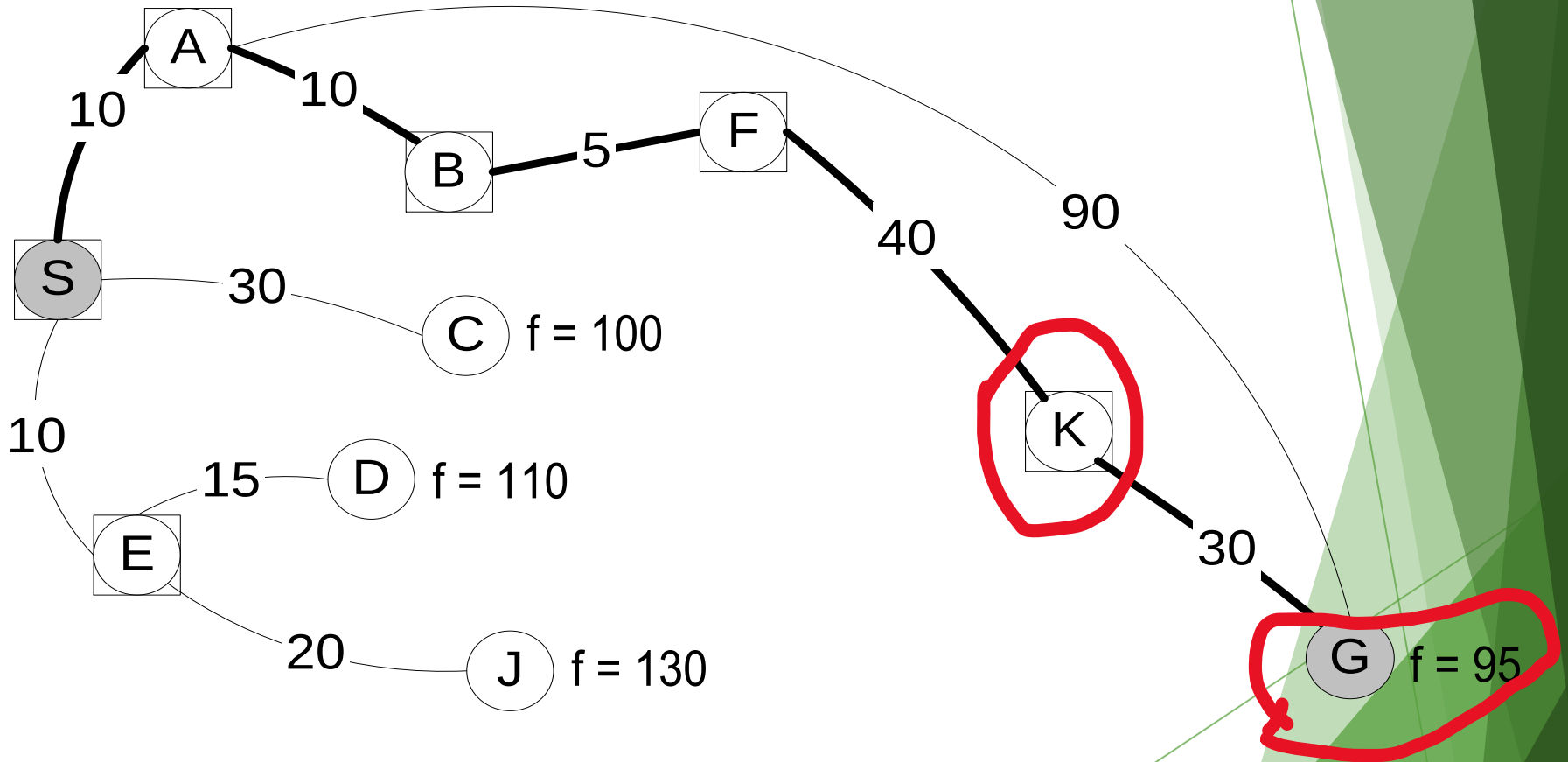
Langkah 5



OPEN=[C, D, G, J]

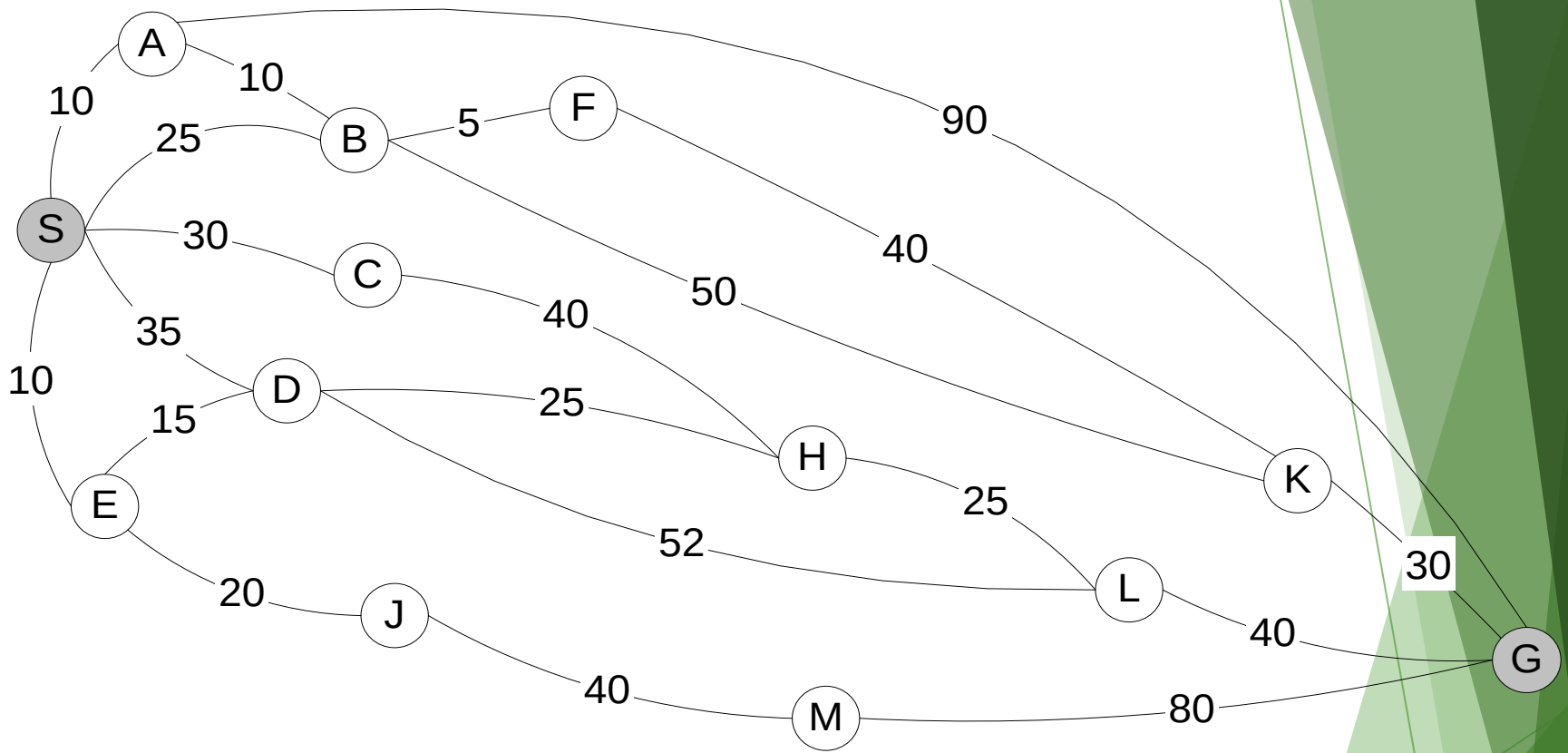
CLOSED=[S, B, E, A, F, K]

Langkah 6



Beam A (BA*)*

- ▶ Membatasi jumlah simpul yang bisa disimpan di dalam *OPEN*.
- ▶ Ketika jumlah simpul di *OPEN* sudah melebihi batas tertentu, maka simpul dengan nilai f terbesar akan dihapus.
- ▶ Sedangkan jumlah simpul di dalam *CLOSED* tetap dibiarkan tanpa batasan karena simpul yang di dalam *CLOSED* memang tidak mungkin dihapus.
- ▶ Dengan membatasi jumlah simpul di *OPEN*, maka pencarian menjadi lebih terfokus seperti sinar (*beam*).



n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

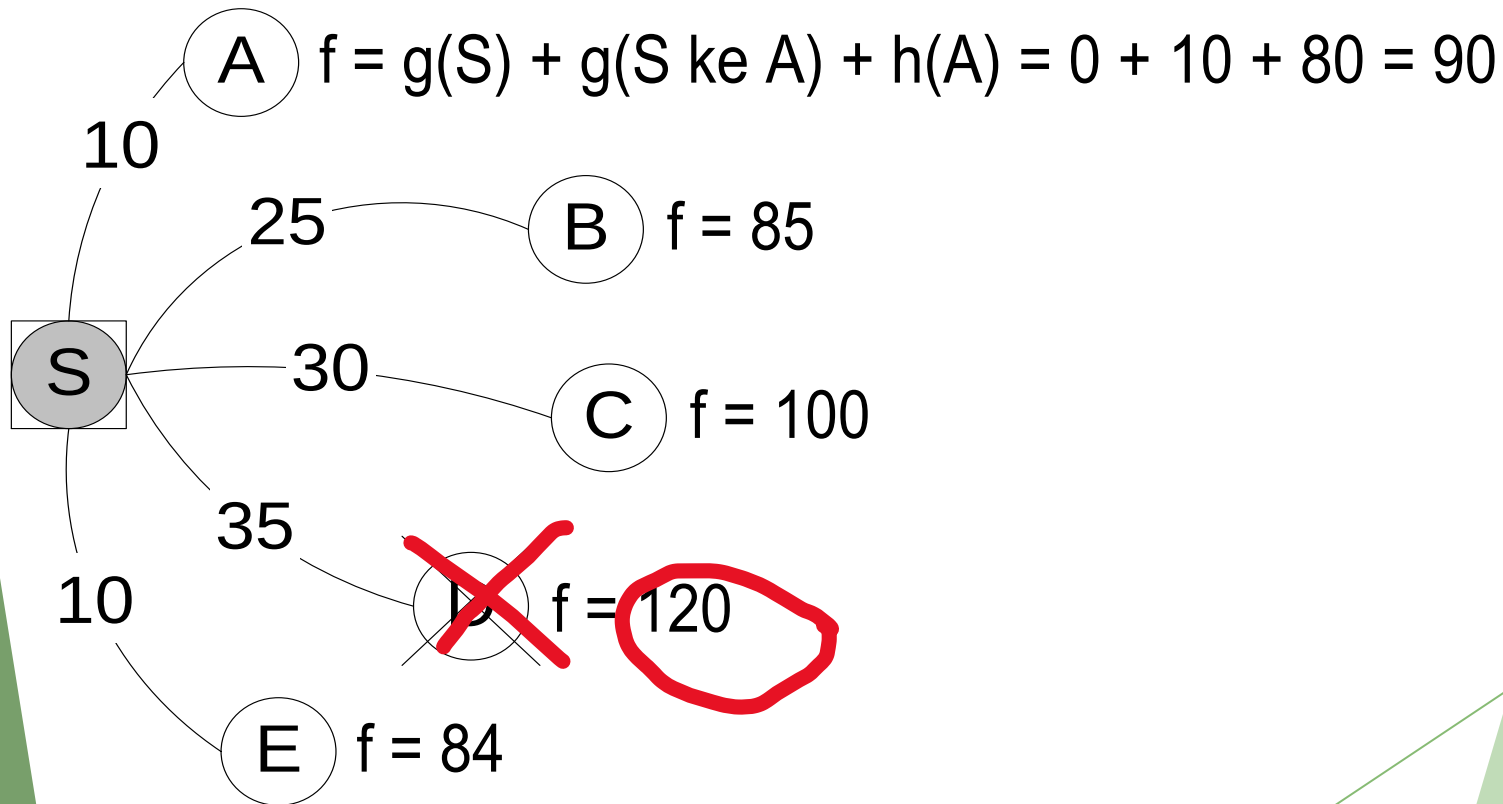
Beam A^* (BA^*)

- ▶ Pada kasus ini, misalkan jumlah simpul maksimum yang bisa disimpan di dalam *OPEN* adalah 4.
- ▶ Bagaimana BA^* menemukan solusi?

OPEN=[A, B, C, E]

CLOSED=[S]

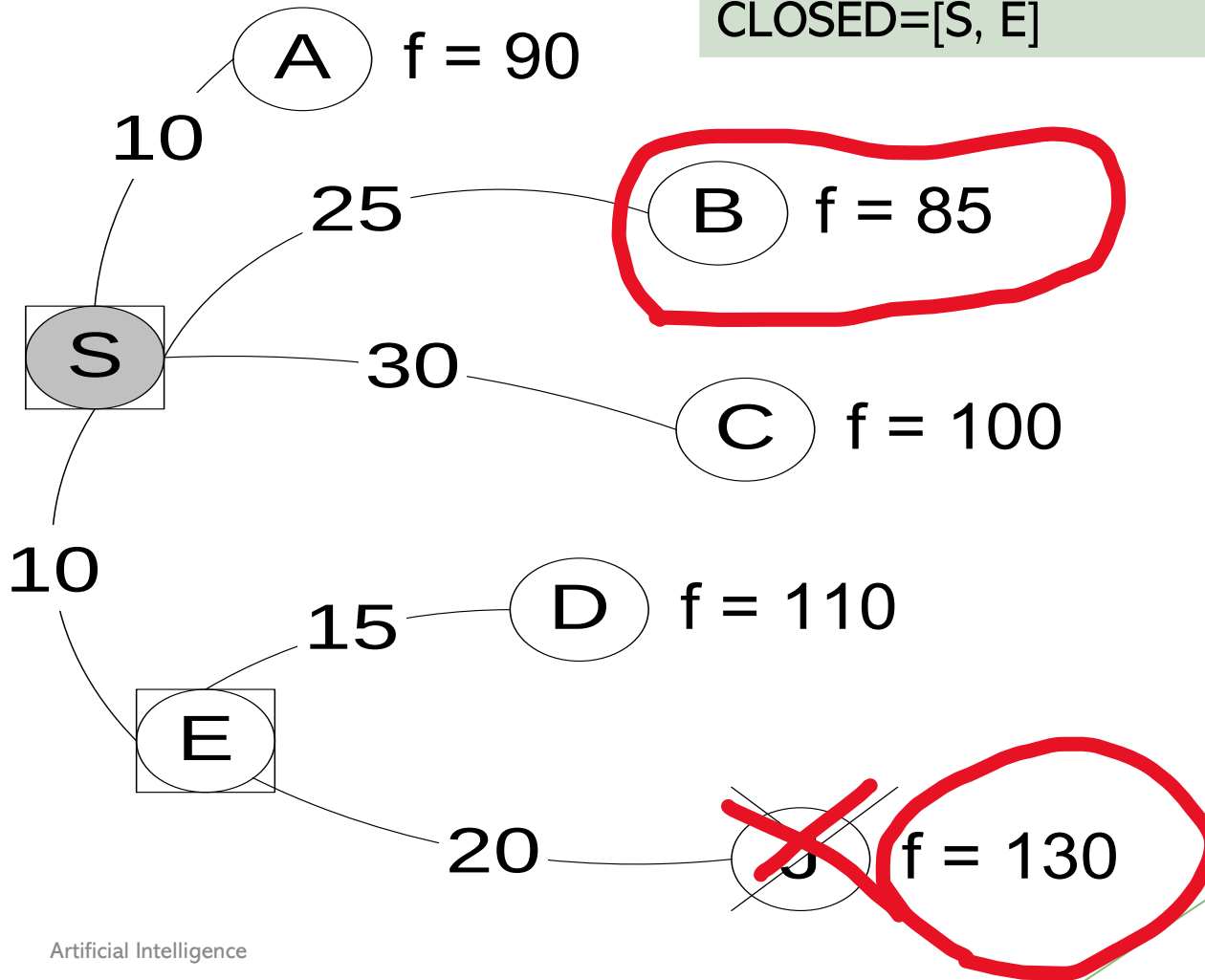
Langkah 1



Langkah 2

OPEN=[A, B, C, D]

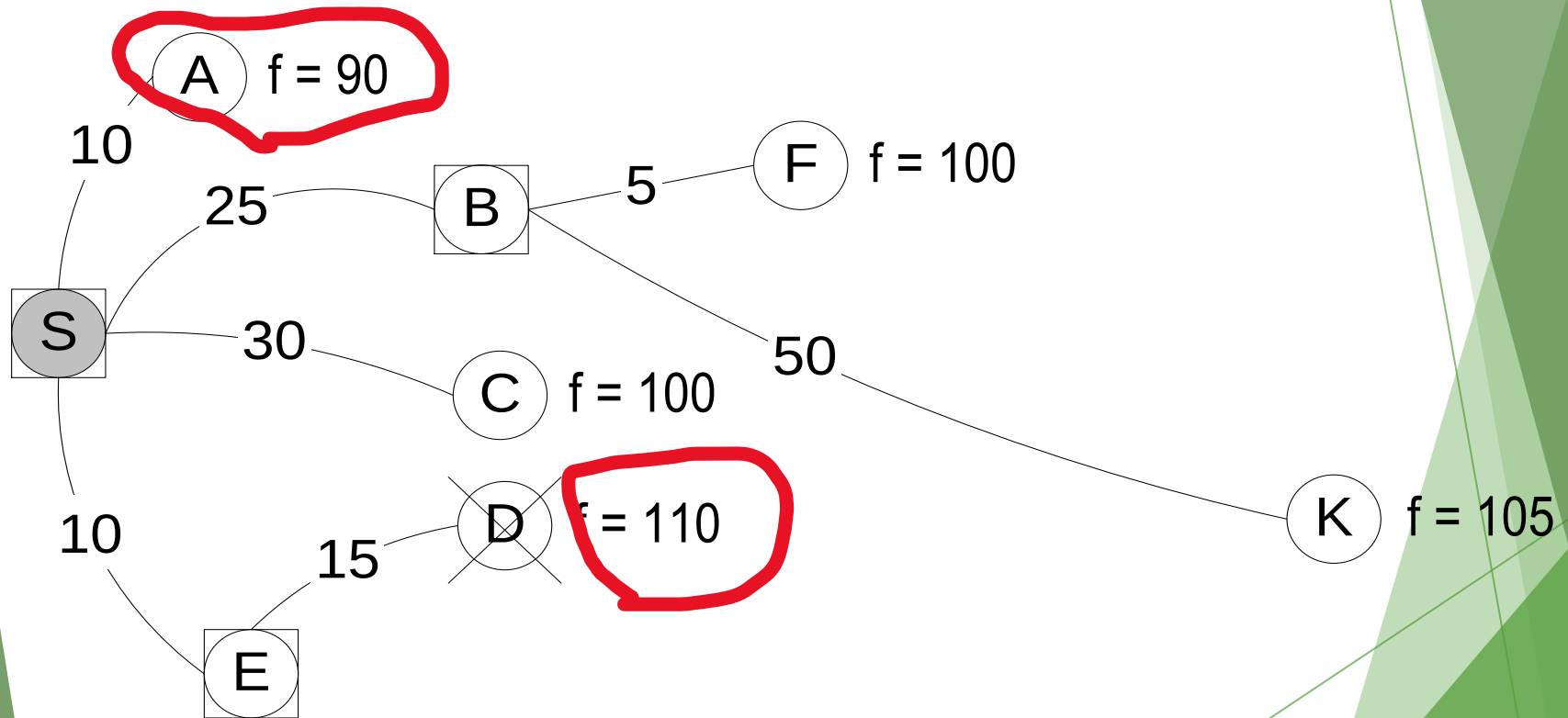
CLOSED=[S, E]



OPEN=[A, C, F, K]

CLOSED=[S, E, B]

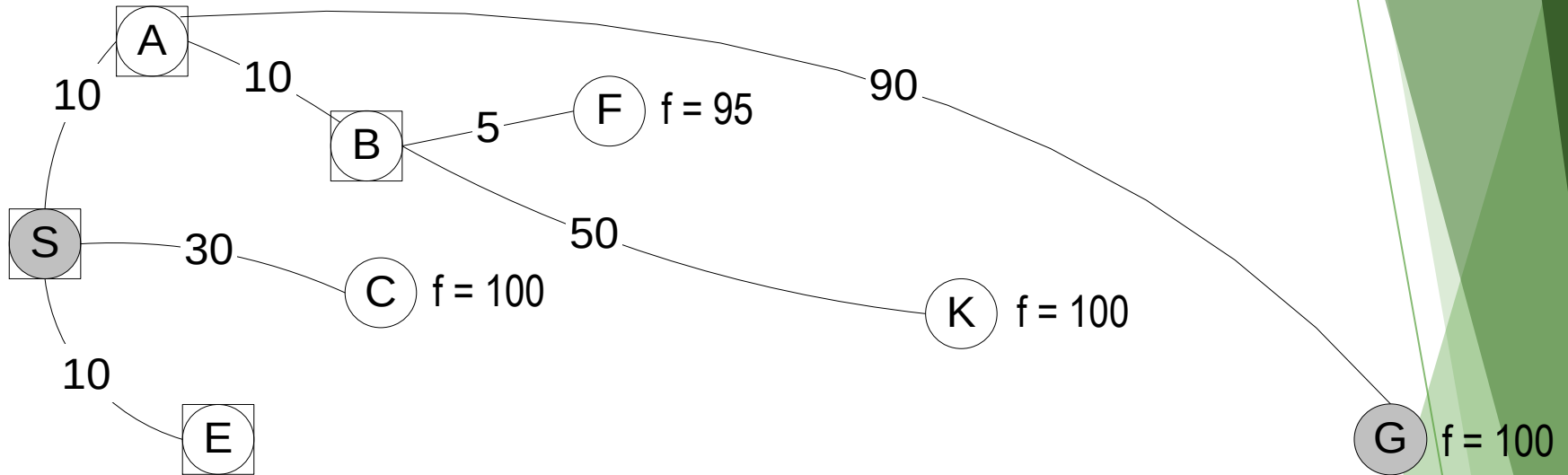
Langkah 3



OPEN=[C, F, K, G]

CLOSED=[S, E, B, A]

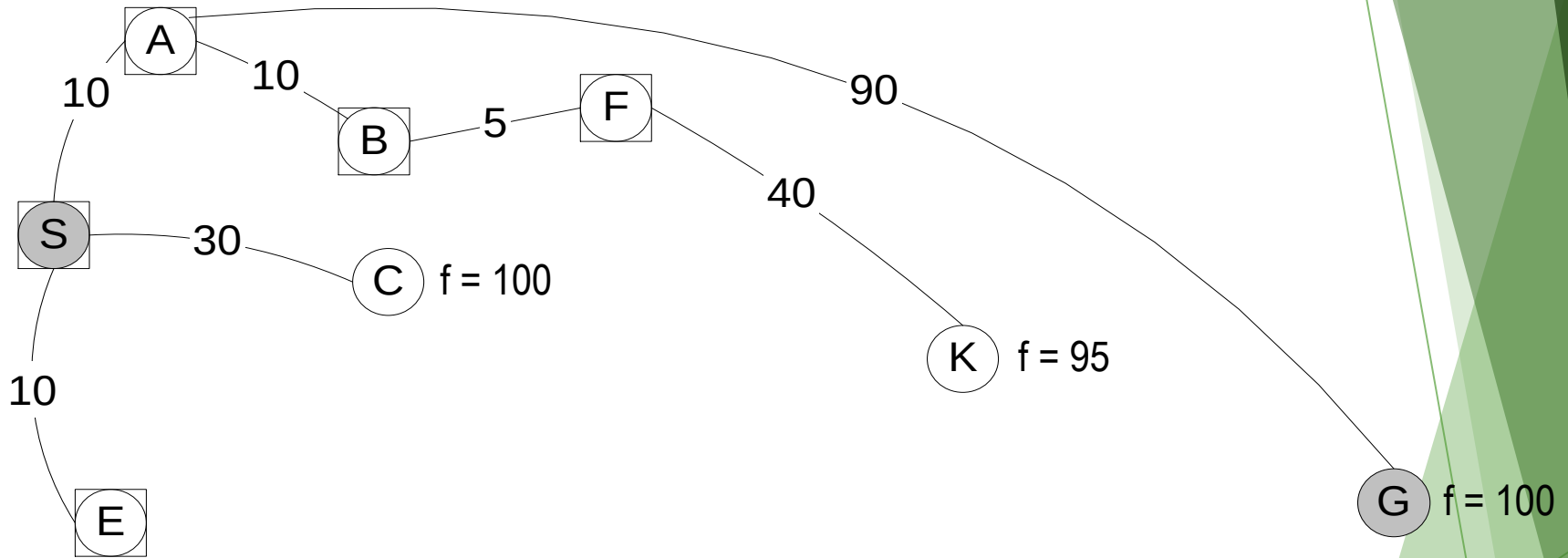
Langkah 4



OPEN=[C, K, G]

CLOSED=[S, E, B, A, F]

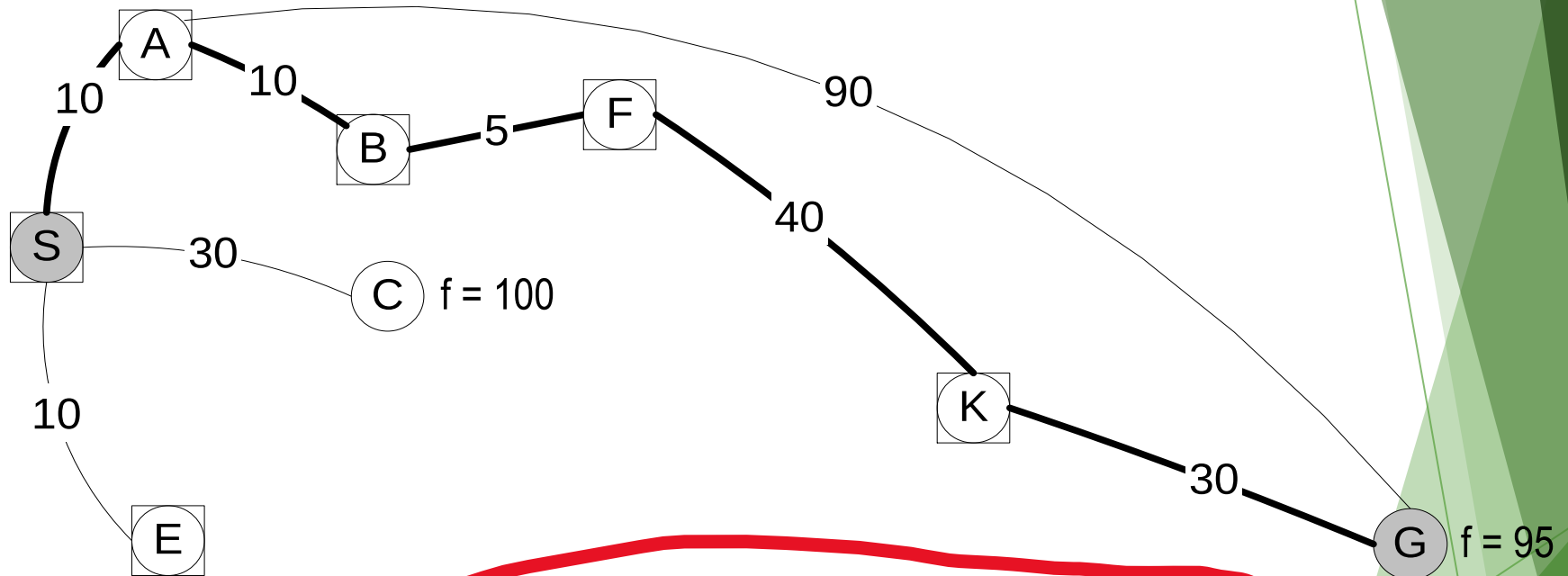
Langkah 5



Langkah 6

OPEN=[C, G]

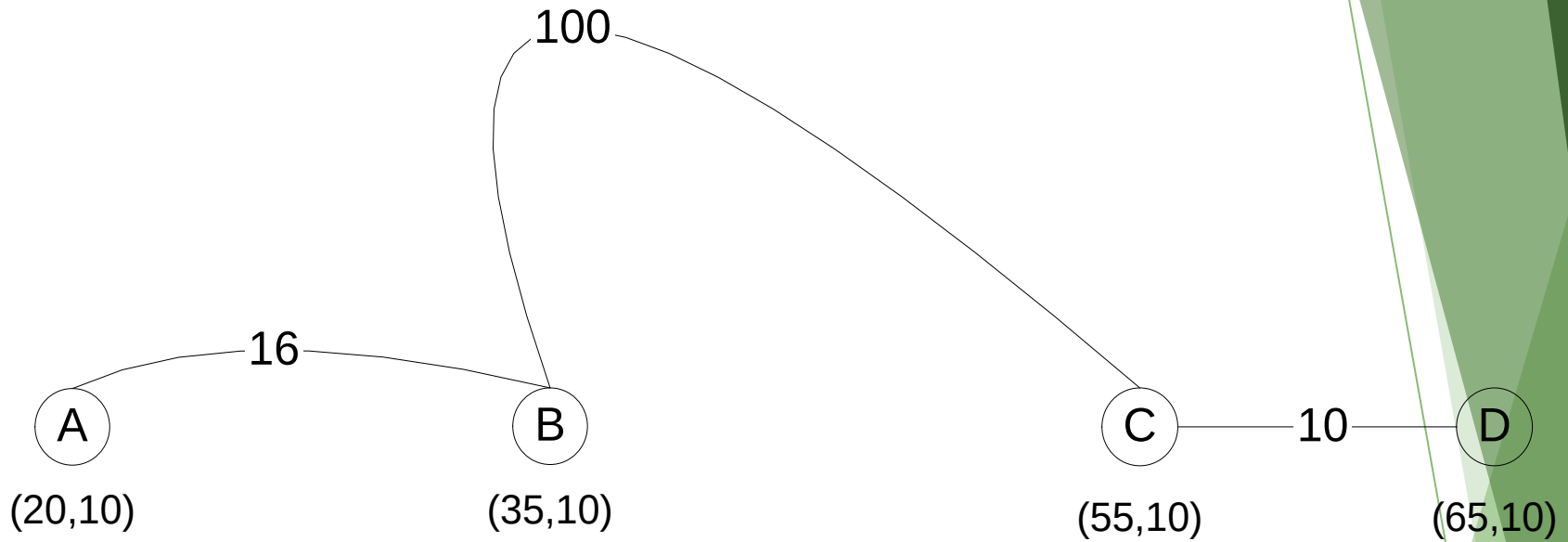
CLOSED=[S, E, B, A, F, K]



HASIL : S-E-B-A-F-K, TOTAL-JARAK=95

Fungsi heuristik

- ▶ Suatu fungsi dapat diterima sebagai fungsi heuristik jika biaya perkiraan yang dihasilkan **tidak melebihi** dari biaya sebenarnya.
- ▶ Jika fungsi heuristik *overestimate*, maka proses pencarian bisa tersesat dan **tidak optimal**.
- ▶ Suatu fungsi heuristik dikatakan baik jika bisa memberikan biaya perkiraan yang **mendekati** biaya sebenarnya.
- ▶ Semakin mendekati biaya sebenarnya, fungsi heuristik tersebut semakin baik.



Fungsi heuristik h berupa jarak garis lurus:

$$d_{ab} = \sqrt{(y_b - y_a)^2 + (x_b - x_a)^2}$$

Masalah 8-puzzle

1	2	3
8	7	4
6		5

Initial State

1	2	3
4	5	6
7	8	

Goal State

Fungsi heuristik = ?

Fungsi heuristik 8-Puzzle

- ▶ h_1 = jumlah kotak yang posisinya salah.
- ▶ Pada masalah di atas, angka 1, 2 , dan 3 sudah berada pada posisi yang benar.
- ▶ Sedangkan lima angka yang lain berada di posisi yang salah.
- ▶ Jadi $h_1 = 5$.

Fungsi heuristik 8-Puzzle

- ▶ h_2 = jumlah langkah yang diperlukan masing-masing kotak menuju posisi yang benar di *goal state*.
- ▶ Biasanya disebut *City Block distance* atau *Manhattan distance*.
- ▶ Untuk masalah di atas, angka 1, 2 dan 3 membutuhkan 0 langkah.
- ▶ Angka 4, 5, 7, dan 8 membutuhkan 2 langkah.
- ▶ Sedangkan angka 6 membutuhkan 3 langkah.
- ▶ Sehingga $h_2 = 0 + 0 + 0 + 2 + 2 + 3 + 2 + 2 = 11$.

Perbandingan IDS dengan A^* yang menggunakan h_1 dan h_2 .
 Data yang digunakan adalah rata-rata 100 kasus 8-puzzle dengan
 kedalaman solusi yang bervariasi [RUS95].

d	Jumlah node yang dibangkitkan			Faktor percabangan		
	IDS	$A^* (h_1)$	$A^* (h_2)$	IDS	$A^* (h_1)$	$A^* (h_2)$
2	10	6	6	2,45	1,79	1,79
4	112	13	12	2,87	1,48	1,45
6	680	20	18	2,73	1,34	1,3
8	6.384	39	25	2,8	1,33	1,24
10	47.127	93	39	2,79	1,38	1,22
12	364.404	227	73	2,78	1,42	1,24
14	3.473.941	539	113	2,83	1,44	1,23
16	-	1.301	211	-	1,45	1,25
18	-	3.056	363	-	1,46	1,26
20	-	7.276	676	-	1,47	1,27
22	-	18.094	1.219	-	1,48	1,28
24	-	39.135	1.641	-	1,48	1,26

Kesimpulan

- ▶ Metode-metode yang termasuk dalam *blind search* membutuhkan memori yang sangat besar untuk menyelesaikan masalah yang sederhana.
- ▶ Dengan kecepatan dan memori komputer yang terbatas, saat ini *blind search* belum memungkinkan untuk diimplementasikan ke dalam dunia nyata.
- ▶ Satu-satunya metode yang mungkin bisa digunakan adalah *Iterative Deepening Search* (IDS) karena membutuhkan memori yang sangat sedikit meskipun waktu prosesnya sangat lama.

Kesimpulan

- ▶ *Simulated Annealing* (SA) merupakan algoritma yang **selalu maju** (pada setiap iterasi selalu menghasilkan simpul di level yang lebih dalam).
- ▶ SA bekerja dengan hanya memperhitungkan *sibling* (simpul yang memiliki *parent* yang sama) sehingga tidak pernah mundur untuk mencari jalur yang lebih baik.
- ▶ Ketika memori dan kecepatan prosesor sangat terbatas dan solusi yang diharapkan tidak harus optimal, maka *Simulated Annealing* (SA) adalah pilihan yang paling tepat di antara semua algoritma *heuristic search*.

Kesimpulan

- ▶ Diantara metode-metode pencarian yang termasuk dalam *heuristic search*, **A* adalah pilihan yang terbaik ketika kita dapat menemukan sebuah fungsi heuristik untuk masalah yang akan diselesaikan.**
- ▶ Kita dapat memilih variasi A* yang paling sesuai dengan masalah yang akan diselesaikan dan sumber daya (waktu dan memori) yang kita miliki.
- ▶ Ketika ditemukan lebih dari satu jenis fungsi heuristik, pilihlah yang **paling mendekati biaya sebenarnya.**
- ▶ Ketika memori yang tersedia sangat terbatas, misalkan pada PDA (*Personal Digital Assistant*), maka variasi-variasi A*, seperti IDA* dan SMA*, dapat digunakan sebagai alternatif lain yang sangat menjanjikan.
- ▶ Tetapi, perlu diperhitungkan jumlah memori dan kecepatan prosesor yang ada.

Kesimpulan

- ▶ Untuk masalah yang memiliki **satu jenis goal**, maka **bi-directional search** bisa dianggap sebagai alternatif terbaik ketika ruang masalahnya sangat besar.
- ▶ *Modified Bi-directional A** (MBDA*) memberikan performansi sangat baik untuk ruang masalah yang sangat besar.
- ▶ *Modified Bi-directional* ini juga bisa menggunakan variasi A* yang lain, misalnya *Beam A** maupun *Weighting A** sehingga kita bisa menyebutnya sebagai *Modified Bi-directional Beam A** atau *Modified Bi-directional Weighting A**.

Daftar Pustaka

- Russel, Stuart and Norvig, Peter. 2010. Artificial Intelligence: A Modern Approach. Prentice Hall International, Inc.
- Suyanto. 2007. Artificial Intelligence: Searching, Reasoning, Planning and Learning. Informatika, Bandung Indonesia. ISBN: 979-1 153-05-1.